ESPRIT

Field: IT

Final-Year Project Report

CST LePoint System

An intelligent platform for transport management, attendance tracking, and real-time monitoring

Student	Farhani Rayen
Supervisor	Mr Bouslahi Hamdi
Academic year	2025–2026

Acknowledgments

I would like to express my deepest gratitude to my academic supervisor and my professional supervisor at CST (Canadian System Technology) for their invaluable guidance, constant support, and constructive feedback throughout this final-year project.

My sincere thanks also go to all the teachers and members of the jury for their time, evaluation, and interest in this work. Finally, I want to thank my family and friends for their encouragement and support during my studies.

Abstract

This report describes the design and implementation of **CST LePoint**, a full-stack, enterprise-grade platform built for the IT service company **CST (Canadian System Technology)** to digitize and optimize fleet tracking and worker attendance management. The solution solves the real-world operational challenges of manual transport rosters and lack of real-time visibility by offering real-time bus tracking via IoT modems, geofenced route-deviation notifications, automated biometric/RFID attendance checking, and interactive business intelligence (BI) layouts.

Furthermore, the system integrates a machine learning microservice built with FastAPI to predict Estimated Time of Arrival (ETA), duration of operations, and absence-risk assessment. Built using a robust technology stack including Angular 18, Tailwind CSS, .NET 8 (following Clean Architecture, CQRS with MediatR, and Carter module routing), and SQL Server, this system enforces multi-tenant security, strict data invariants, and containerized deployment via Docker.

Keywords: Fleet Management, Attendance Tracking, Clean Architecture, CQRS, Machine Learning, FastAPI, Angular, IoT Telemetry.

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Full Meaning
API	Application Programming Interface
BI	Business Intelligence
CQRS	Command Query Responsibility Segregation
CST	Canadian System Technology
EF Core	Entity Framework Core
ETA	Estimated Time of Arrival
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JWT	JSON Web Token
ML	Machine Learning
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
RFID	Radio-Frequency Identification
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
ULID	Universally Unique Lexicographically Sortable Identifier
UML	Unified Modeling Language
WSL	Windows Subsystem for Linux

General Introduction

Over the past decade, the rapid advancement of the Internet of Things (IoT), cloud computing, and artificial intelligence has revolutionized traditional industrial management workflows. In the domain of corporate logistics, transportation systems have transitioned from passive support services into highly integrated, data-driven operations. For large enterprises operating in industrial zones—such as manufacturing plants, automotive assembly facilities, and call centers—organizing the daily transit of thousands of workers across multiple shifts is a complex operational challenge.

Ensuring that employees arrive on time directly impacts production line efficiency, labor relations, and organizational cost control. Traditionally, companies managed these services using fragmented, manual workflows. Roster scheduling was planned on paper or spreadsheets, attendance checks were conducted manually by drivers, and real-time fleet positions were completely invisible. This lack of integration created significant operational bottlenecks: delayed routes went unnoticed, driver identity could not be verified in real time, and invoicing subcontractor fleets was prone to accounting discrepancies.

Report Plan

This thesis documents the complete engineering process of the CST LePoint platform, from requirements analysis to architectural design, implementation, and deployment. The report is structured into five core chapters:

1. **Chapter 1: Project Overview** introduces the host company, Canadian System Technology (CST), details the operational problems, outlines the target user personas, and defines the technical scope and activities of the project.
2. **Chapter 2: Study of Existing Solutions** critiques legacy fleet tracking and biometric gate solutions, evaluates the baseline manual workflows, and presents a concrete operational scenario illustrating the financial impact of transport delays.
3. **Chapter 3: Requirements Specification** defines the functional and non-functional requirements of the platform, presents the use case models (UC-1 through UC-6), and summarizes functional traceability.
4. **Chapter 4: Proposed Solution** details the system design, highlighting Clean Architecture layers, Domain-Driven Design (DDD) concepts, strongly-typed ULID identifiers, database schemas, and MediatR pipeline behaviors.
5. **Chapter 5: Implementation** presents the technology stack, agile Scrum sprint achievements, detailed C# and TypeScript code listings, Docker Compose configuration scripts, and quality assurance testing routines.

Finally, a General Conclusion summarizes the contributions of the project and outlines future extensions for AI-assisted logistics in corporate transport networks.

1 Project Overview

1.1 Introduction

In modern industrial and logistics-intensive organizations, employee transportation has evolved from a simple secondary service into a core operational dependency. For enterprises operating with multiple work shifts, distributed factories, or extensive call center networks, the coordination of staff transit directly impacts productivity, labor satisfaction, and operational budgets. Meeting these demands requires real-time visibility, automated attendance tracking, route compliance auditing, and intelligent planning tools.

The **CST LePoint** platform was initiated to address these challenges by centralizing vehicle fleet management, employee pickup routing, real-time geofenced trace supervision, and introducing an artificial intelligence layer to predict bus Estimated Time of Arrival (ETA) and assess employee absenteeism risks. This chapter establishes the project's background, detailing the host company, the operational problem statement, target user personas, technical objectives, and scope boundaries.

1.2 Host Company

The host company for this project is **Canadian System Technology (CST)**, a pioneering enterprise specializing in Information Technology services, systems integration, and software development. Founded in Montréal, Canada, in 1989 by Mr. Samir Gara, Canadian System Technology established its primary subsidiary in Tunisia in 1997. Over the past three decades, the company has built its reputation on engineering enterprise-grade solutions, consolidating its capabilities through the fusion of four specialized technology entities:

- **GIC**: The developer and publisher of the *Le Point* Enterprise Resource Planning (ERP) suite. The ERP comprises several specialized modules, including:
  - *Le Point-GPAO*: Computer-Aided Production Management (Gestion de Production Assistée par Ordinateur), optimizing industrial manufacturing flows.
  - *Le Point-GPM*: Maintenance Management (Gestion des Processus de Maintenance), tracking equipment lifecycles and interventions.
  - *Le Point-GESTCOM*: Commercial Management (Gestion Commerciale), automating purchasing, sales, and inventory operations.
  - *Le Point-FICO*: Financial and Accounting Management (Gestion Financière et Comptable), handling general ledger, tax filing, and cash flow.
  - *Le Point-GRH*: Human Resources Management (Gestion des Ressources Humaines), managing personnel records, biometric pointage, payroll, and shifts.
- **GIC-Distribution**: An infrastructure systems integrator focusing on high-availability and security solutions. Its key areas of expertise include:
  - Corporate cybersecurity and network access control.
  - Systems consolidation, server virtualization, and high-performance computing clusters.
  - Enterprise storage, backup strategies, and disaster recovery planning.
- **Bs4m**: A specialized publisher of mobile business solutions. Bs4m focuses on bridging physical field activities with centralized enterprise logic, offering technologies in:
  - Mobile Point of Sale (POS) and retail management.
  - Mobile field service intervention and ticketing.
  - Fleet tracking, telemetry, and geolocation services.
- **Canadian Software Technology**: A software development and management consulting firm leveraging cutting-edge web and system frameworks to design custom systems

for production planning, integrated ERPs, and mobile banking.

The **CST LePoint** platform represents a strategic integration within GIC's software ecosystem. By combining the human resource and scheduling principles of GIC-GRH with the real-time telemetry and geofencing capabilities developed by Bs4m, the platform delivers a unified system to manage both personnel attendance and fleet geolocation.

1.3 Project Context and Existing System

1.3.1 Industrial Transport Logistics in Tunisia

Many Tunisian industrial enterprises (such as automotive wire-harness factories, textile plants, and call centers) operate 24/7 across three consecutive 8-hour shifts. Because these factories are often located in industrial zones far from residential districts, companies must organize transport networks to pick up thousands of employees from various pickup points daily. This requires managing dozens of rented or corporate buses, planning optimized paths, and verifying that employees board their designated vehicles.

1.3.2 The Traceability Challenge

Before the initiation of CST LePoint, coordination between dispatchers, drivers, and corporate HR departments relied almost entirely on manual processes, leading to critical inefficiencies. There was no real-time geolocation of buses, attendance checks were performed using manual paper logs, and drivers' routes could not be verified for compliance.

1.3.3 CST LePoint Legacy Platform (Existing Solution)

Before the implementation of the new web-based and intelligent CST LePoint platform, the host company deployed a legacy desktop application to manage transit scheduling. This existing solution suffered from significant limitations:

- **Desktop-Only Access**: The application was a fat-client desktop installation, restricting access to designated office workstations. Dispatchers could not monitor operations on the field or from home, and employees had no visibility into bus locations or schedules.
- **Barebone Operations**: The platform served primarily as a static registry for bus

circuits and driver contacts. Geolocation and route compliance were handled retrospectively, with no real-time tracking or automatic alert systems.

- **No Predictive Analytics**: There was no capability to forecast arrival times (ETA) or identify potential worker absenteeism. It was entirely reactive, depending on phone calls to determine bus delays.
- **Simple Business Intelligence**: Reporting was basic and static, showing only simple aggregate figures without interactive dashboards, custom layouts, or multi-dimensional analytics.

1.3.4 Development Environment and Deployment Stack

To resolve these inefficiencies, our internship project introduces a modernized web-based architecture. A high-level overview of the technological stacks for both systems includes:

- **Legacy System**: A fat-client desktop application directly connected to a centralized database server. Deployment was restricted to local office terminals, and data collection from the field was manually transcribed from paper sheets.
- **New Web Platform**: An Angular 18 frontend Single Page Application (SPA) utilizing Tailwind CSS and Leaflet maps, communicating with a .NET 8 (ASP.NET Core) backend Web API.
- **Machine Learning Service**: A FastAPI Python microservice running alongside the backend to provide predictive analytics (ETA and absenteeism risks).
- **Deployment Stack**: Containerized using Docker Compose to host the frontend (via Nginx), backend API, SQL Server database, and the FastAPI ML service in isolated environments. IoT modems onboard the buses send real-time coordinates and attendance data via the SMQTT protocol.

1.4 Project Description

The primary goal of our internship project is to replace the legacy desktop platform with this intelligent, web-based transport and attendance management system. The platform will automate tracking, verify route compliance, and leverage machine learning to provide predictive metrics for dispatchers and commuters.

1.4.1 Target User Personas

The simplified system boundaries involve four core user roles:

- **Administrators**: Manage global configurations, tenant settings, and security roles.
- **Operators**: Monitor active routes, driver assignments, and receive real-time geofence alerts.
- **Analysts**: Generate business intelligence reports, audit telemetry traces, and customize reporting layouts.
- **Commuters (Employees)**: Access the web client to view their shift schedules, routes, and live bus positions.

1.4.2 Operational Problems and Risk Boundaries

Developing and deploying a real-time, telemetry-intensive platform introduces key technical and operational risks:

- **GPRS Connection Dropouts**: Loss of cellular coverage in rural zones, requiring local caching and timestamp audits to ensure data integrity.
- **Machine Learning Service Latency**: Delay in predicting ETA, resolved by using asynchronous parallel calls with deterministic fallback rules.
- **Data Ingestion Volume**: Large volume of telemetry coordinates causing database bottlenecks, mitigated by EF Core query splitting and indexing.
- **Multi-Tenancy Security**: Risks of cross-company data leakage, mitigated by enforcing database-level global tenant filters.

1.5 Conclusion

This chapter presented the host company context, the legacy CST LePoint desktop application, the development stack, the deployment architectures for both the legacy and new systems, and the overall objectives of the new project.

However, the legacy system still poses several unresolved operational difficulties. The next chapter, *Study of the Existing*, will further discover the existing solution in detail, analyzing its limitations, the financial and operational impact of those limitations, and the identified rooms for improvement.

2 Study of Existing Solutions

2.1 Introduction

In corporate transport and workforce dispatching logistics, managing fleet assets and tracking personnel presence have historically been treated as distinct administrative concerns. This fragmentation results in operational inefficiencies, security vulnerabilities, and high manual overhead. This chapter reviews the standard commercial systems currently used in the market, analyzes the manual coordination processes replaced by CST LePoint, evaluates a concrete operational failure scenario, and critiques the specific limitations of these legacy solutions.

2.2 Analysis of Commercial Solutions

To evaluate the current state of the art, we analyze three categories of commercial systems commonly deployed in fleet tracking and personnel pointage:

2.2.1 Dedicated Fleet Geolocation Systems (Wialon & GPSGate)

Wialon and GPSGate are market-leading commercial platforms designed for vehicle telematics and fleet tracking. They operate by receiving coordinates and diagnostic indicators from physical GPS modems (such as Teltonika or Ruptela devices) installed in vehicles, parsing these packets, and rendering real-time markers on web-based maps.

- **Strengths**: Highly mature cartographic engines, robust support for geofenced speed alerts, fuel level monitoring, and generic travel history reports.
- **Limitations**: These platforms are completely isolated from corporate databases. They operate without any knowledge of who is inside the vehicle. For employee transit, this means dispatchers cannot verify whether the correct operators have boarded, nor can

the system associate driver shifts or track absent workers. The lack of integration with corporate HR directories prevents these systems from solving the employee transit tracking problem.

2.2.2 Biometric Gate Terminals (ZKTime & stationary scanners)

Stationary biometric pointage systems (such as ZKTime terminals, fingerprint turnstiles, and facial recognition devices) are installed at factory gates or main office entrances to audit employee arrival times.

- **Strengths**: High verification accuracy, resistance to buddy-punching (credential sharing), and direct integration with local payroll and HR scheduling modules.
- **Limitations**: These systems suffer from a severe operational blind spot. They only record when an employee crosses the office gate. If a company-provided bus is delayed, is stuck in traffic, or breaks down, the HR system simply flags the employees as late or absent. Dispatchers and plant managers have no way of knowing if the missing workers are on their way, on which vehicle they are riding, or if they are still waiting at their pickup points. This lack of mobility tracking leads to delayed scheduling decisions and production bottlenecks.

2.2.3 Manual Coordination and Communication

For many local small-to-medium enterprises, transportation monitoring is handled using basic communication platforms (such as paper list sheets, phone coordination, and SMS or WhatsApp dispatch groups).

- **Strengths**: Zero capital cost and immediate setup.
- **Limitations**: Manual tracking does not scale. Drivers must manually check off names while operating vehicles, creating safety risks. Paper log sheets are frequently lost, illegible, or subject to falsification. Cross-referencing monthly paper sheets with contractor invoices requires significant administrative labor and is highly prone to calculation errors.

2.3 Current Dependency Tracking Process

The manual tracking workflow replaced by CST LePoint follows five distinct stages:

1. **Weekly Rostering**: Every Friday, HR administrators create employee transport assignments in Excel sheets, linking workers to specific routes and shifts.
2. **Roster Distribution**: Transport coordinators print paper lists and distribute them to drivers.
3. **Manual Boarding Check**: As workers enter the bus, the driver verifies identity card details and manually checks off names.
4. **Status Communication**: In the event of traffic jams or vehicle failures, drivers must call the dispatcher to explain their location verbally.
5. **Invoice Auditing**: At the end of the month, paper rosters are transcribed into spreadsheets. Administrators cross-reference these sheets with invoices from external transit providers to calculate mileage, fuel allowances, and driver wages.

The sequence of operational steps and communication channels for this manual tracking workflow is depicted in Figure 2.1.

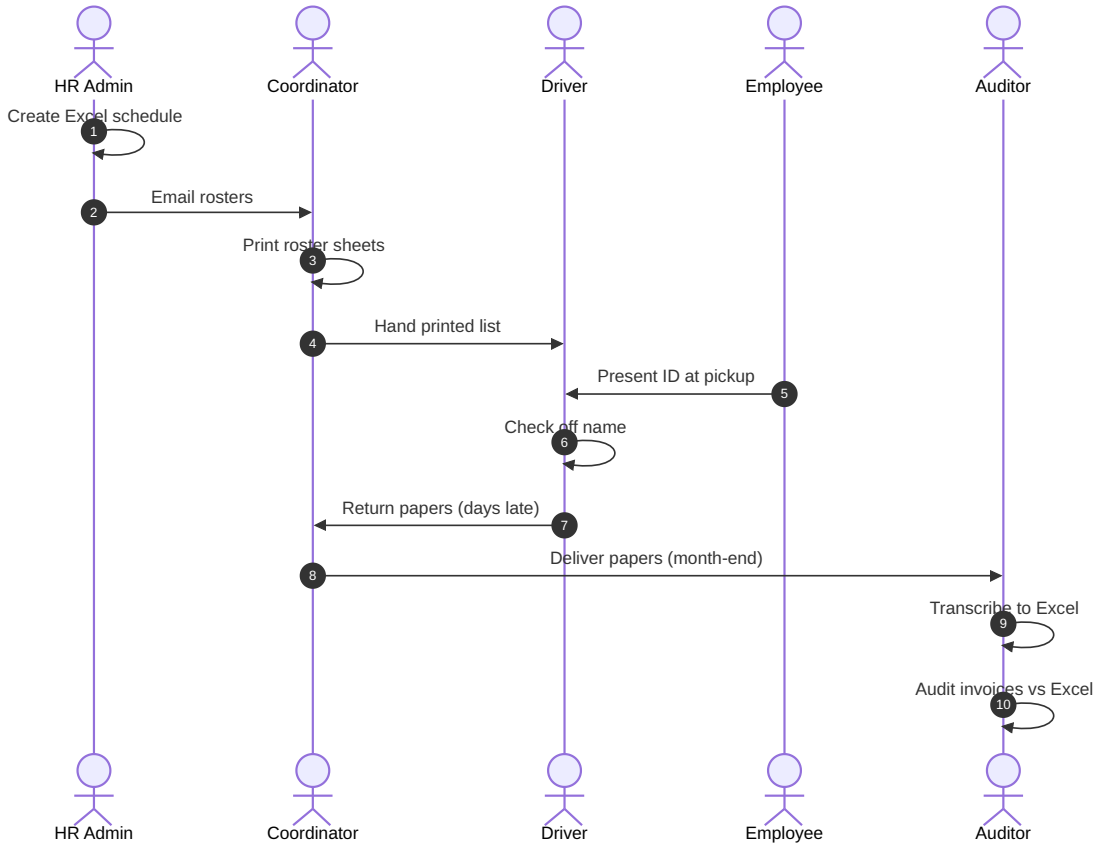

Figure 2.1: Manual Boarding and Attendance Process Sequence

2.3.1 Legacy Deployment Architecture

In the legacy system, the technological architecture was restricted to a centralized database server with local office desktop fat-client instances. Information about transit execution remained isolated in physical logbooks and driver notes, with data manually entered long after the events occurred. Figure 2.2 illustrates the legacy desktop-to-database flow and the lack of real-time network integration with the active fleet.

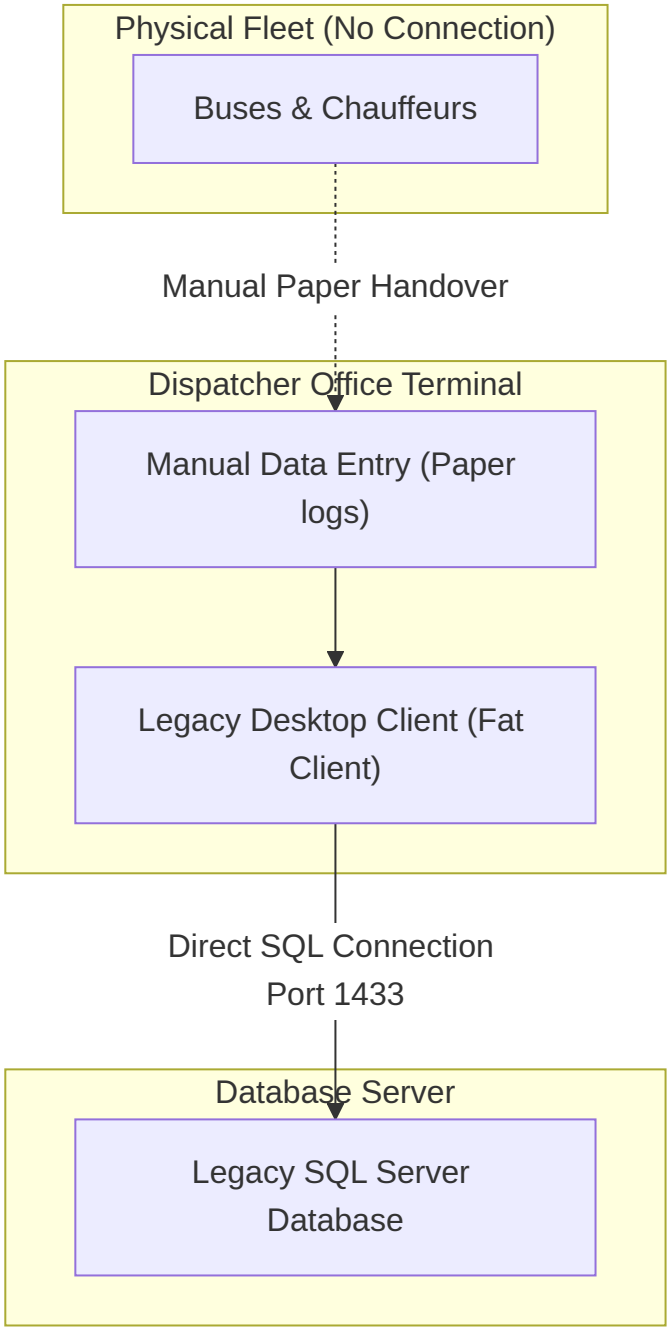

Figure 2.2: Legacy Deployment Architecture and Data Flow

Criteria	Manual Logbook	GPS Tracking	Office Biometrics	CST LePoint
HR Database Link	None	None	Direct (Gate only)	Fully Integrated
Live Geolocation	None	Yes	None	Yes + Geofencing
Boarding Audit	Paper sheets	None	None	Biometric/RFID check
Route Audit	Visual inspect	Basic traces	None	Automatic geofencing
Delay Predictions	None	None	None	ML ETA (FastAPI)
Security & RBAC	None	Weak	Strong	Granular RBAC
Operational Cost	High labor	Medium (License)	Low	Low (Automated)

Table 2.1: Comparative Matrix of Transport and Attendance Tracking Solutions

2.4 Financial and Operational Impact of Transport Delays

To understand the necessity of an integrated system, we must evaluate the financial and operational consequences of transit failures in industrial manufacturing environments:

2.4.1 The Assembly Line Vacancy Problem

In automotive parts assembly or electronic manufacturing plants operating on lean manufacturing models, production lines require a strict complement of operators to run safely. If a bus carrying 40 operators is delayed by 30 minutes, the assembly line cannot run at full speed. This leads to immediate bottlenecks, idle workstations downstream, and delayed product shipments.

2.4.2 Concrete Delay Calculation

Consider a standard Tunisian automotive wiring-harness factory with an hourly production value of $15,000$ TND.

- A bus breakdown occurs, delaying 40 specialized operators for 1.5 hours.
- Because the operators are missing, a primary production line is halted.
- The financial cost of this halt is calculated as:

  $$\text{Loss} = \text{Hourly Production Value} \times \text{Duration} + \text{Overtime Compensations} = 15,000 \times 1.5 + 2,500 = 25,0$$

- In a manual environment, the plant manager is notified only after the shift starts, leaving no time to reassign workers from other lines.

By contrast, an automated real-time tracking system identifies when a vehicle stops moving on a scheduled route, evaluates the geofence breach, calculates the delayed ETA, and alerts dispatchers within 5 minutes, allowing managers to adjust shift patterns proactively.

2.5 Detailed Operational Scenario

Consider a morning shift breakdown scenario: At 6:15 AM, a corporate bus carrying 40 factory operators scheduled for the 7:00 AM shift breaks down on a secondary highway.

- **Roster State**: No live GPS track is active. The driver tries to call the dispatcher, but the line is busy.
- **Search and Rescue**: When the driver finally connects at 6:40 AM, they cannot provide exact coordinates. The dispatcher must call other vehicles to locate the broken bus.
- **Result**: The replacement bus reaches the site at 7:20 AM and delivers the workers to the factory at 7:50 AM, causing a major assembly line shutdown.

The sequence of events and communication breakdowns during this scenario is illustrated in Figure 2.3.

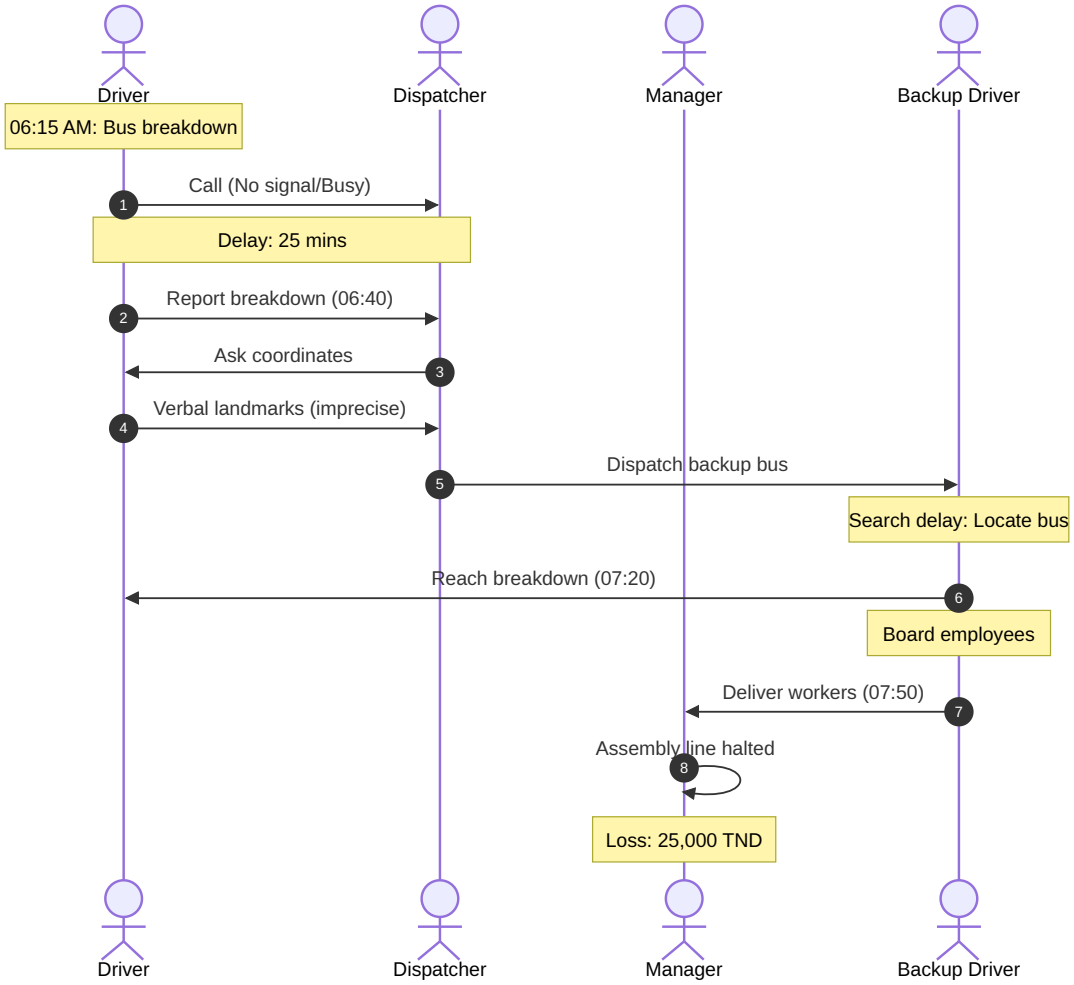

Figure 2.3: Communication and Logistical Timeline during a Vehicle Breakdown

2.6 Limitations and Their Impact

The critique of the manual and fragmented approach highlights several structural, operational, and technical limitations. These constraints directly degrade the logistical efficiency of industrial transit operations, causing significant financial waste, security vulnerabilities, and excessive administrative overhead.

2.6.1 Detailed Technical and Operational Limitations

The legacy system suffered from four fundamental operational and technical limitations:

- **Desktop-Only Constraints**: The legacy application was deployed as a desktop fat-client directly wired to the SQL Server database. This localized setup meant that

transit scheduling was only accessible from office workstations. Field dispatchers could not update schedules or driver details dynamically from the yard, managers had no remote operational visibility, and employees had no way to check bus schedules or routes.

- **Manual Paper Rosters**: Attendance check-ins relied entirely on printed sheets that drivers manually filled during boarding. This method is highly vulnerable to environmental damage (such as water or tearing), illegible handwriting, and data discrepancies. Furthermore, the attendance records remained in the vehicle for days or weeks before being manually transcribed, creating a massive latency in HR record updates.
- **Lack of Real-Time Geolocation**: With no real-time telemetry, the dispatcher office was completely blind to vehicle positions. The system served as a static database of planned circuits and driver contacts. If a bus was delayed, coordinators could only estimate its location and status by making direct phone calls to the driver, which is both dangerous while driving and prone to subjective reports.
- **Absence of Route Compliance Auditing**: There was no telemetry feedback loop to verify if drivers followed the scheduled circuits. Drivers could skip pickup points, take unauthorized detours, or deviate from planned paths to avoid traffic without the system registering any anomaly. This lack of auditing made it impossible to enforce route compliance or optimize transit loops.

2.6.2 Business and Financial Impact on Logistics

These technical limitations translate directly into measurable business and financial losses for the company:

- **Delayed Shifts and Assembly Line Vacancy**: Tunisian manufacturing plants, such as automotive wiring-harness factories, operate on strict lean production models where workstations are tightly sequenced. A delay of a single bus carrying 30 or 40 technicians results in vacant assembly stations, slowing or halting the entire downstream line. The financial cost of such delays is severe, as calculated in the breakdown scenario ($25,000$ TND in idle labor and overtime recovery costs for a 1.5-hour delay).
- **Fuel Wastage and Unmonitored Mileage**: In the absence of automated route auditing, external transport contractors frequently bill for routes that are longer than necessary or routes they did not fully complete. Excessive engine idling, detours for driver personal errands, and unoptimized paths cause substantial fuel wastage and inflated subcontractor billing.

- **Security and Liability Risks**: Manual rosters do not offer secure verification of passengers. If a traffic accident or roadside emergency occurs, the company cannot quickly determine which employees were aboard the vehicle, creating serious regulatory and safety compliance liabilities. Furthermore, paper rosters are insecure and can easily be accessed by unauthorized personnel, exposing sensitive employee schedules and locations.
- **Administrative Burden and Invoice Auditing Delay**: At the end of every monthly billing cycle, administrative teams must manually type thousands of paper check-in entries into Excel files and cross-reference them with subcontractor invoices. This manual verification takes dozens of hours, is highly prone to human mathematical errors, and delays payments to transit providers.

2.7 Identified Rooms for Improvement and Transition

To resolve these operational difficulties and secure its logistics chain, Canadian System Technology requires a modernized, unified platform. The transition from the legacy desktop client to a cloud-ready web and mobile framework must focus on three core areas of improvement:

1. **Real-Time Telemetry and Cartographic Supervision**: Replace static database registries with an active, web-based map interface. Using IoT modems installed in the buses, the system must process real-time coordinates via standard protocols (such as MQTT or HTTP) and dynamically plot vehicle positions, allowing dispatchers to monitor route progress and catch delays instantly.
2. **Automated Attendance Tracking**: Replace manual paper sheets with digital boarding checks using RFID card scanners linked directly to the on-board IoT modems. As employees tap their company cards, the system must instantly validate their credentials, log their boarding status, and update the centralized HR database in real time.
3. **Predictive Analytics and Intelligence**: Incorporate a machine learning layer to move from reactive scheduling to proactive management. The system should automatically query historical transit data and traffic conditions to calculate accurate Estimated Times of Arrival (ETA) for commuters, and analyze attendance traces to predict absenteeism risks before shifts begin.

2.8 Conclusion

This chapter detailed the existing system and the limitations of manual transportation tracking workflows. It demonstrated the operational and financial impact of delays on lean manufacturing lines and identified key areas for improvement, establishing that a web-based, real-time, and predictive system is necessary to replace the legacy fat-client.

To implement this modernized system, a comprehensive set of specifications must be defined. The next chapter establishes the functional and non-functional requirements specification for the proposed CST LePoint platform.

3 Requirements Specification

3.1 Introduction

Defining precise requirements is essential to building an enterprise-grade solution that aligns technical implementations with operational realities. This chapter details the functional and non-functional requirements, presents the global and sub-system use case models, and establishes the boundaries of the CST LePoint system.

3.2 Functional Requirements

The functional requirements cover all the actions that the system must support. To align these requirements with modern software engineering practices, they are organized into core operational modules and detailed as User Stories:

3.2.1 Authentication and Authorization

- **FR-01: Secure Sign-In** (Sprint 1, 3 SP, Priority: High)
  As a User, I want to authenticate with credentials and SocieteId so that I can access my company scope.
- **FR-02: Stateless Token Generation** (Sprint 1, 2 SP, Priority: High)
  As a User, I want to receive a signed JWT token upon login so that my session is authenticated without server-side state.
- **FR-03: Token Revalidation** (Sprint 1, 1 SP, Priority: Medium)
  As a Client App, I want to verify the token validity on startup so that I can automatically restore active sessions.
- **FR-04: Granular Access Control** (Sprint 2, 5 SP, Priority: High)
  As an Administrator, I want to assign navigation and API permissions to roles so that

users only see and execute authorized actions.

3.2.2 Fleet Management

- **FR-05: Bus Registration** (Sprint 2, 2 SP, Priority: High)
  As an Administrator, I want to manage buses (immatriculation, model, capacity) so that they can be assigned to routes.
- **FR-06: Driver Assignment** (Sprint 2, 2 SP, Priority: High)
  As an Operator, I want to link drivers to buses and RFID badges so that operational attendance can be registered.
- **FR-07: Modem Association** (Sprint 2, 2 SP, Priority: Medium)
  As an Administrator, I want to catalog IoT modems and pair them with buses so that telemetry can be mapped to active vehicles.

3.2.3 Pickup Planning

- **FR-08: Circuit Definition** (Sprint 2, 3 SP, Priority: High)
  As an Operator, I want to create circuits with GPS coordinates, distance, and duration so that routes can be planned.
- **FR-09: Ordered Pickup Points** (Sprint 2, 3 SP, Priority: High)
  As an Operator, I want to sequence pickup points along a circuit so that the bus follows a strict logistical route.

3.2.4 Passenger and Shift Management

- **FR-10: Employee Directory** (Sprint 2, 3 SP, Priority: High)
  As an Administrator, I want to manage employee records, RFID tags, and geographic coordinates so that pickups can be targeted.
- **FR-11: Shift Configuration** (Sprint 2, 2 SP, Priority: Medium)
  As an Administrator, I want to define working shifts (start/end times) so that employees can be scheduled on transit routes.
- **FR-12: Operational Work Orders** (Sprint 2, 3 SP, Priority: Medium)
  As an Operator, I want to schedule site work orders for teams so that logistical transport matches factory demand.

3.2.5 IoT Telemetry Ingestion

- **FR-13: Real-Time Telemetry Ingestion** (Sprint 4, 5 SP, Priority: High)
  As a Modem, I want to submit periodic GPS/RFID occupancy payloads to the API so that fleet status is updated in real-time.
- **FR-14: Telemetry Replay Protection** (Sprint 4, 2 SP, Priority: High)
  As a System, I want to reject telemetry payloads older than 15 minutes so that replay attacks are prevented.
- **FR-15: Automatic Geofence Tracking** (Sprint 4, 5 SP, Priority: High)
  As a System, I want to flag a bus detour if it deviates more than 250m from the route so that deviations are logged immediately.

3.2.6 Real-Time Supervision

- **FR-16: Cartographic Tracking** (Sprint 3, 5 SP, Priority: High)
  As an Operator, I want to view active buses moving in real-time on a map so that I can supervise fleet routes visually.
- **FR-17: Telemetry Timeline** (Sprint 3, 3 SP, Priority: Medium)
  As an Operator, I want to see a chronological log of events for each bus so that I can analyze runtime alerts and updates.

3.2.7 Predictive Analytics

- **FR-18: ETA Forecasting** (Sprint 4, 5 SP, Priority: High)
  As an Operator, I want to receive machine-learning predicted ETAs for active buses so that I can inform plants of potential delays.
- **FR-19: Batch ETA Calculations** (Sprint 4, 3 SP, Priority: High)
  As a Backend, I want to query FastAPI for ETAs of all active buses in parallel so that tracking updates remain efficient.

3.2.8 Business Intelligence Reporting

- **FR-20: Dynamic Querying & Reporting** (Sprint 5, 5 SP, Priority: High)
  As an Analyst, I want to run historical telemetry and attendance reports with sorting, grouping, and exporting so that I can run audits.

- **FR-21: Custom Report Layouts** (Sprint 5, 3 SP, Priority: Medium)
  As an Analyst, I want to save my custom column groupings and configurations so that I can reload them with one click.

3.3 Non-Functional Requirements

The non-functional requirements describe the quality attributes, technological standards, and operational constraints achieved in the CST LePoint project:

3.3.1 Performance and Ingestion Latency

To handle high-frequency data generated by the active fleet, the system must adhere to strict performance targets:

- **Ingestion Response Time**: Telemetry ingestion API requests (`POST /cm/bus/runtime/positions`) must be processed and resolved in under 200ms to avoid network queues on cellular IoT devices.
- **High Concurrency Support**: The backend must support concurrent telemetry updates from 50+ IoT modems broadcasting GPS/RFID packet data every 30 seconds without database deadlock or memory leaks.
- **UI Grid Load Latency**: Historical datagrids rendering thousands of telemetry trace records must complete client-side sorting, pagination, and filtering in less than 500ms using virtual scrolling and optimized EF Core query parameters.

3.3.2 Scalability and Containerization Boundary

The system architecture must support horizontal scaling as the company's fleet size grows:

- **Container Separation**: Services must be isolated into independent container layers (Frontend client, .NET Core WebAPI, SQL Server 2019, and FastAPI ML prediction service) orchestrated using Docker Compose.
- **Stateless API Layers**: The backend application is stateless, allowing multiple instances of the .NET WebAPI container to run behind a reverse proxy (Nginx) to distribute concurrent REST traffic.

3.3.3 Data Integrity and CQRS Transactional Boundaries

Operational logs and financial reports demand absolute data consistency:

- **Unit of Work Pattern**: Database mutations must be bound by transactional boundaries managed via MediatR Pipeline Behaviors. A failure in any handler step must trigger an automatic transaction rollback.
- **Fail-Fast Input Validation**: Before committing modifications to the persistence layer, incoming data must pass strict FluentValidation checks at the application commands boundary.

3.3.4 Security, Cryptography, and Privacy Standards

As an enterprise-grade platform, security controls are configured at both transmission and storage levels:

- **Stateless Authentication**: All client-server communication must be authenticated using JSON Web Tokens (JWT) signed via HMAC-SHA256, carrying expiration times and client metadata claims.
- **Credential Hashing**: Passwords must never be stored in plain text. They are hashed using BCrypt, utilizing a configurable work factor to protect against brute-force attacks.
- **Action-Level Authorization**: System operations are restricted based on user role permissions (Read, Add, Edit, Delete) mapped dynamically to application routing modules.

3.3.5 System Maintainability and DDD Architectural Styles

To ensure long-term code quality and reduce technical debt:

- **Clean Architecture Rules**: Core business logic (Domain and Application layers) must remain pure, with zero dependencies on external frameworks, SGBDs, or third-party mapping libraries.
- **Domain Invariants Enforcements**: Domain models must use private setters, requiring state changes to flow through explicit constructor factories and validation methods.
- **ULID Indexing**: Distributed, lexicographically sortable ULIDs (Universally Unique Lexicographically Sortable Identifiers) are used as primary database keys to optimize

clustering indexing over classic random GUIDs.

3.3.6 Robustness and Fault Tolerance

The platform must degrade gracefully when external dependencies fail:

- **ML Service Failure Handling**: If the FastAPI machine learning container becomes unreachable, the WebAPI backend must fallback to static duration heuristics based on deterministic circuit distances, logging a warning while keeping the user experience uninterrupted.
- **Replay Attack Defenses**: The ingestion engine must validate telemetry packet timestamps, rejecting any payload delayed by more than 15 minutes to protect against data injection.

3.4 Use Case Model

To model the interactions between users, external systems, and the CST LePoint platform, we establish a formal use case model. We present the global use case diagram first, followed by the definitions of actors, external systems, detailed use case descriptions, and sub-system diagrams.

3.4.1 Global Use Case Diagram

The global use case diagram illustrates the complete scope of the platform, linking human actors and external systems to the core administrative, tracking, predictive, and analytical capabilities of the system.

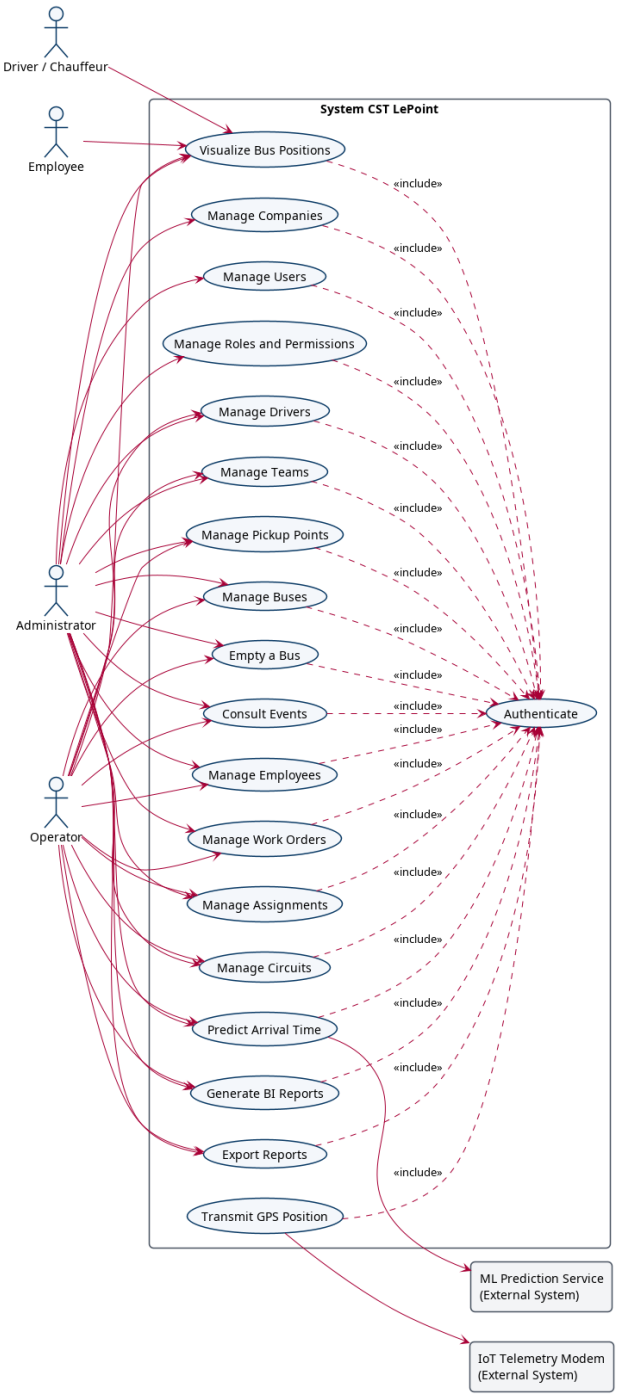

Figure 3.1: Global Use Case Diagram – CST LePoint Platform

3.4.2 Actors

The system defines the following primary human actors:

- **Administrator**: Manages security configurations, users, roles, and master data catalogs.

- **Operator / Dispatcher**: Supervises active routes, assigns buses to drivers, and monitors operational alerts.
- **Analyst**: Evaluates historical geolocation traces, runs operational reports, and customizes BI dashboards.
- **Employee**: Accesses the application to view real-time bus locations, schedule information, and verify which vehicle they are assigned to for transport.
- **Driver / Chauffeur**: Accesses the application to view vehicle details and verify their bus assignments.

3.4.3 External Systems

The system interacts with the following external systems:

- **IoT Telemetry Modem**: A physical device installed onboard the buses that automatically sends GPS coordinates and passenger occupancy counts to the API.
- **ML Prediction Microservice**: An external FastAPI service that processes input features to generate ETA predictions and absence-risk evaluations.

3.4.4 Use Case Specifications

This section details the primary use cases developed in the project, providing their formal flows and preconditions:

UC-1: Authenticate User

- **Primary Actors**: Administrator, Operator, Analyst, Employee, Driver.
- **Preconditions**: The user account is active, and the client application is loaded at the login screen.
- **Postconditions**: The user session is initialized, a JWT Bearer token is saved in the browser, and the authorized navigation menu is rendered.
- **Main Flow**:
  1. The user enters their username, password, and company reference (`SocieteId`).
  2. The user clicks the "Sign In" button.

3. The client application validates the inputs locally and sends an HTTP POST request containing credentials to the backend login route.
4. The WebAPI receives the payload and dispatches a login command through the MediatR bus.
5. The command handler hashes the input password and compares it against the stored BCrypt hash.
6. The handler fetches the user's role and associated navigation permission matrix.
7. The system generates a cryptographically signed JWT token containing user identity and permission scopes.
8. The API returns an HTTP 200 response wrapping the token and profile metadata.
9. The client application saves the JWT token to local storage and updates the current session state.
10. The router redirects the user to the default landing dashboard.

- **Alternative Flows**:
  - *Invalid Credentials*: If the username or password does not match, the system aborts, returning an HTTP 401 response. The client displays a warning notification and keeps the user on the login screen.
  - *Inactive Account*: If the user account is disabled (`IsActive = false`), the system returns an HTTP 403 response, preventing login.

UC-2: Track Fleet Geolocation

- **Primary Actors / Systems**: IoT Telemetry Modem, Operator.
- **Preconditions**: The vehicle is active, equipped with an initialized modem, and the operator is logged into the tracking board.
- **Postconditions**: The vehicle position and occupancy are updated in the database, and the new marker state is broadcast to the map UI.
- **Main Flow**:
  1. The onboard IoT modem collects GPS coordinates and passenger counts.
  2. The modem transmits an HTTP POST request to the API's telemetry route.
  3. The WebAPI validates that the coordinate timestamp falls within a 15-minute replay-protection buffer.
  4. The system maps the incoming IMEI code to the corresponding bus entity.

5. The API dispatches the update command through MediatR, initiating a database transaction.
6. The handler calculates the distance between the vehicle position and scheduled route points.
7. The handler updates the current latitude, longitude, occupancy, and timestamps on the bus record.
8. The repository commits the database changes, closing the transaction.
9. The system publishes a telemetry update event to the WebSocket channel.
10. The client's Leaflet map receives the WebSocket payload and moves the bus marker to the new coordinates.

- **Alternative Flows**:
  - *Geofence Route Detour*: If the distance between the bus and scheduled route points exceeds 250 meters, the system automatically instantiates a new pickup point and records a geofence alert event before committing.
  - *Unrecognized IMEI*: If the IMEI does not map to any registered bus, the API rejects the request with an HTTP 404 response.

UC-3: Request ETA Prediction

- **Primary Actors**: Operator / Dispatcher.
- **Preconditions**: The dispatcher is logged in, and a bus is actively operating on a scheduled circuit.
- **Postconditions**: The predicted arrival time and model confidence score are rendered on the tracking board.
- **Main Flow**:
  1. The dispatcher selects an active bus on the monitoring map.
  2. The client application requests the current telemetry values of the bus.
  3. The backend API receives the query and aggregates current bus variables (coordinates, speed, occupancy, capacity, and circuit ID).
  4. The API generates an HTTP prediction request payload.
  5. The backend HttpClient calls the FastAPI ML microservice endpoint on port 8001.
  6. The FastAPI microservice loads the trained model artifacts, runs the prediction logic, and calculates the ETA in minutes along with a confidence metric.

7. The FastAPI service returns the prediction results over HTTP.
8. The backend WebAPI receives the response and wraps it in a standard API contract.
9. The frontend tracking panel receives the results and displays the minutes remaining for the bus to reach its destination.

- **Alternative Flows**:
  - *FastAPI Service Offline*: If the Python microservice is unreachable, the backend API logs a critical warning and returns a default estimated duration based on deterministic circuit distances.

UC-4: Consult Assignment and Bus Status

- **Primary Actors**: Employee, Driver.
- **Preconditions**: The employee or driver is authenticated and holds standard read-only access.
- **Postconditions**: The user's specific bus assignment details, schedules, and active locations are rendered on screen.
- **Main Flow**:
  1. The employee or driver logs into the mobile-responsive web client.
  2. The application reads the user identity and resolves their employee matricule or driver code from the JWT claims.
  3. The frontend sends a GET request to retrieve the user's specific assignment metadata.
  4. The backend database queries the active assignments (`Rattachement`) table to resolve matching shifts, circuits, and buses.
  5. The API returns the details of the assigned bus (e.g. driver name, circuit code, immatriculation number).
  6. The client client displays the vehicle details, schedule timings, and active route paths.
  7. The user views the real-time position of the assigned vehicle on a Leaflet map.
- **Alternative Flows**:
  - *No Active Assignment*: If the user is not assigned to any route or bus for the current date, the application displays a friendly message stating that no active transportation roster is scheduled.

UC-5: Manage Roles and Permissions

- **Primary Actors**: Administrator.
- **Preconditions**: The Administrator is authenticated and has navigated to the Roles and Permissions module.
- **Postconditions**: A security role is registered or updated in the database with custom navigation-to-action mapping.
- **Main Flow**:
  1. The Administrator requests the list of security roles.
  2. The system retrieves active roles and displays them in a DevExtreme grid.
  3. The Administrator clicks "Add Role" or selects an existing role to edit.
  4. The system opens the role details panel, rendering a checkbox grid of application modules (e.g. `fichier.circuit`, `monitoring.bus-tracking`, `analyse.bi-bus`) and actions (*Read*, *Add*, *Edit*, *Delete*).
  5. The Administrator selects the authorized actions for each module.
  6. The Administrator enters a descriptive name for the role and clicks "Save".
  7. The client application compiles the permission selections into a JSON array and transmits a PATCH/POST request to the backend.
  8. The backend API receives the payload, validates parameters, and dispatches the command.
  9. The command handler updates the `RoleUtilisateur` entity and serializes the permissions list into the database column.
  10. The transaction commits, returning an HTTP 200 OK success response to the client.
- **Alternative Flows**:
  - *Duplicate Role Name*: If a role with the same name already exists in the same company context, the API returns a validation error, and saving is aborted.

UC-6: Save BI Report Layout

- **Primary Actors**: Analyst.
- **Preconditions**: The Analyst is authenticated and has customized a BI grid layout (column reordering, sorting, or grouping filters).

- **Postconditions**: The layout configuration JSON is persisted in the database and linked to the Analyst's company context.
- **Main Flow**:
  1. The Analyst opens a dynamic BI dashboard (e.g. BI Bus or BI Employee).
  2. The Analyst adjusts grid column layouts (hiding specific fields, reordering headers, or establishing default grouping parameters).
  3. The Analyst clicks the "Save Layout" button.
  4. The client application displays a dialog prompting the Analyst for a configuration name.
  5. The Analyst enters a name (e.g. "Monthly Transit Summary") and submits the dialog.
  6. The client application serializes the current grid layout metadata into a JSON string and sends a POST request to `/cm/analyse/layouts`.
  7. The WebAPI receives the command and dispatches it through MediatR.
  8. The command handler updates or creates a `ReportLayout` aggregate, scoping it by `SocieteId`.
  9. The database commits the changes, returning a confirmation.
  10. The frontend adds the saved layout to the dashboard menu for future one-click loading.
- **Alternative Flows**:
  - *Invalid Layout Name*: If the Analyst submits an empty layout name, the client rejects the submission and displays an inline validation error.

3.4.5 Input Validation Constraints

To maintain data integrity and prevent corrupt persistence writes, the CST LePoint platform enforces strict validation rules at the application boundary using FluentValidation. Inbound requests are checked against the following constraints:

- **Bus Entity Constraints**:
  - `NumeroIMM` (Immatriculation plate format): Required, must not exceed 50 characters, and must follow standard Tunisian plate expressions (regex: `^[0-9]+-TN-[0-9]+$`).
  - `IMEI` (Onboard modem identifier): Required, must be exactly 15 numeric digits.

  - `Capacite` (Vehicle seating capacity): Required, must be an integer between 1 and 150.

- **Employee Entity Constraints**:
  - `Matricule` (HR employee identification): Required, unique, and cannot exceed 20 characters.
  - `RFID` (Badge hardware identifier): Required, must be a valid alphanumeric hexadecimal token up to 50 characters.
  - `Latitude/Longitude` (Home geolocation): Must fall within the geographic bounds of Tunisia (Latitude: $[30.0, 38.0]$, Longitude: $[7.0, 12.0]$).

- **Circuit Entity Constraints**:
  - `CodeCircuit` (Route catalog identifier): Required, unique, alphanumeric code up to 20 characters.
  - `DistanceKm` (Transit path length): Required, must be a positive decimal value greater than 0.0.
  - `DureeMinutes` (Estimated transit duration): Required, must be a positive integer greater than 0.

3.4.6 Sub-System Use Case Diagrams

The diagrams below represent the detailed use case models for specific operational sub-systems:

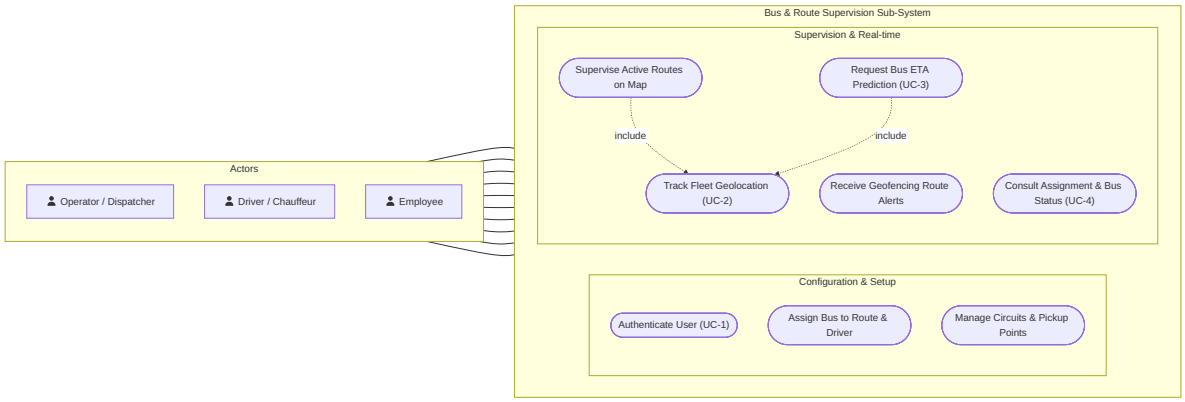

Figure 3.2: Sub-System Use Case – Bus and Route Supervision

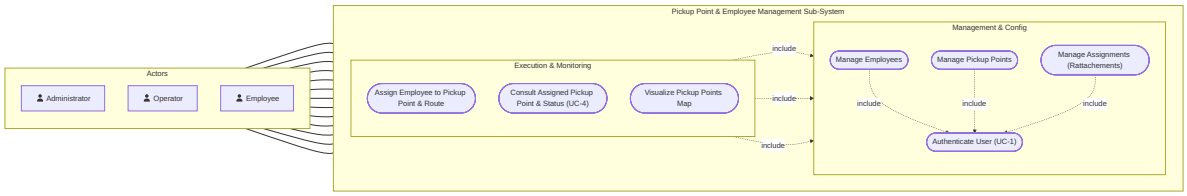

Figure 3.3: Sub-System Use Case – Pickup Point and Employee Management

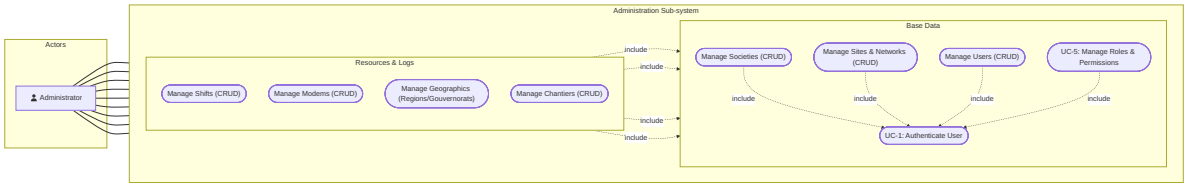

Figure 3.4: Sub-System Use Case – Administration and Security Configurations

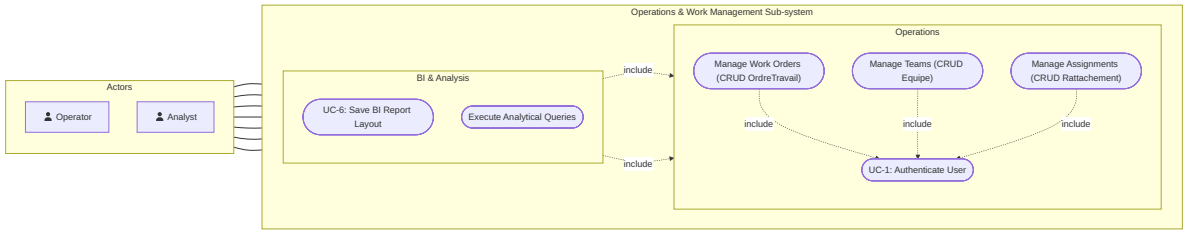

Figure 3.5: Sub-System Use Case – Operations and Work Management

3.4.7 Scope Boundaries

The system boundary separates the software components from physical hardware. The telemetry modems and raw cellular data carriers are treated as external systems. The ML prediction service is run as an external FastAPI container, communicating with the backend over HTTP.

3.5 Requirements Traceability Summary

To ensure complete coverage of the design scope, we cross-reference our functional requirements with the corresponding use cases and implementation features in the table below:

ID	Requirement Name	Traceable Use Cases / Implementation Components
REQ-01	User Authentication	UC-1, BCrypt Hashing, JWT Bearer Token validation
REQ-02	Navigation Permissions	RoleUtilisateur navigation matrices, Navigation Guards
REQ-03	Fleet CRUD Catalogs	Admin dashboard, DevExtreme DataGrids, EF Core context
REQ-04	Telemetry Ingestion	UC-2, minimal API telemetry endpoint, IMEI validation
REQ-05	Live Geofencing alerts	Haversine distance tracking, BusRuntimeEvent logging
REQ-06	Cartographic Map View	ngx-leaflet component, WebSocket polling, marker states
REQ-07	Machine Learning ETA	UC-3, Python FastAPI microservice, PredictBusEta queries
REQ-08	Custom BI Layouts	ReportLayout entity, UpsertReportLayout, ApexCharts

Table 3.1: Functional Requirements Traceability Matrix

3.6 Conclusion

This chapter presented the detailed functional and non-functional requirements of the CST LePoint platform, specified the flows of four core use cases, and mapped these requirements to implementation components. Having defined the system specifications, the next chapter presents the proposed system architecture, database modeling, and logical design solutions.

4 Proposed Solution

4.1 Introduction

This chapter details the comprehensive architectural and technical design of the CST LePoint platform. To overcome the structural limitations of the legacy ecosystem identified in Chapter 2 and fulfill the operational, predictive, and administrative requirements established in Chapter 3, the proposed solution adopts a modern, service-oriented, multi-tier architecture. The platform is engineered to deliver high scalability, strict data integrity, modular maintainability, and real-time fleet visibility for enterprise transit operations.

In this chapter, we focus strictly on the conceptual, architectural, and structural foundations of the system, setting aside specific technology choices and software libraries for Chapter 5.

In the following sections, we systematically present:

- The global multi-tier system architecture and its inter-service communication protocols;
- The physical deployment topology orchestrated through containerized microservices;
- The backend logical architecture governed by Clean Architecture and Domain-Driven Design principles;
- The in-process CQRS request processing pipeline and transactional behavior chain;
- The relational database design, shared-schema multi-tenancy model, and domain schemas;
- The stateless authentication, cryptographic protections, and navigation-based RBAC security model;
- The high-throughput IoT telemetry ingestion pipeline and automated geofencing engine;
- The predictive machine learning microservice integration for real-time bus arrival forecasting;
- The client-side Single Page Application architecture, reactive state management, and internationalization framework.

4.2 Global System Architecture

The platform is structured as a decoupled, multi-tier distributed system composed of four specialized functional layers: the Client Layer, the Backend Layer, the Data Layer, and the Artificial Intelligence (AI) Layer. Each tier fulfills a distinct operational responsibility while communicating over standardized protocols, ensuring loose coupling, independent scalability, and testability.

Figure 4.1 provides an overview of the global system architecture and the communication pathways connecting each subsystem.

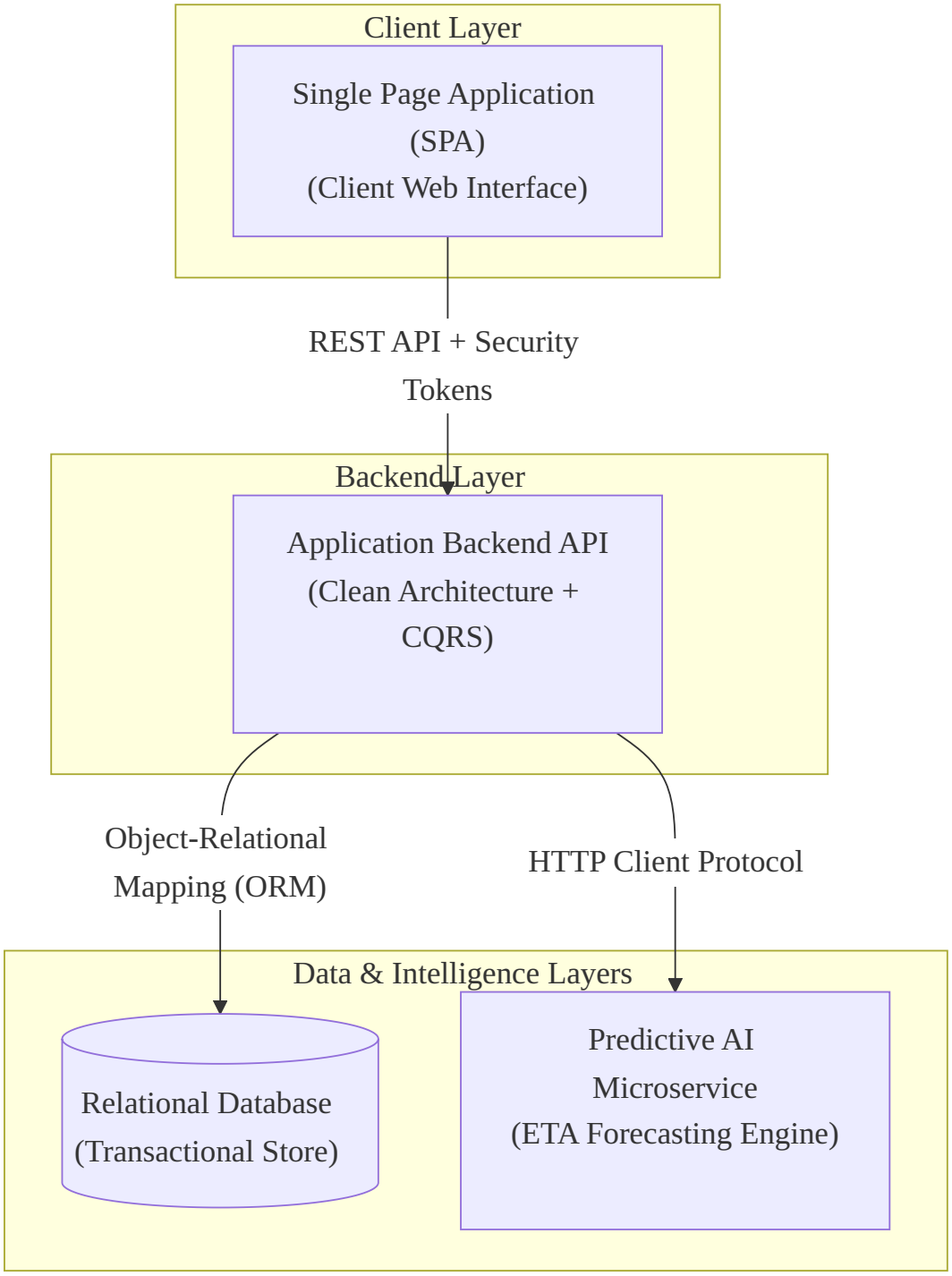

Figure 4.1: Global System Architecture – Multi-Tier Conceptual View

The four architectural tiers operate as follows:

1. **Client Layer (Single Page Application)**: Delivers an interactive, responsive web user interface executed within the end-user's browser. It is served statically through an optimized web server and communicates with the backend exclusively via asynchronous

RESTful HTTP calls carrying signed cryptographic security tokens for authentication and authorization.

2. **Backend Layer (Application Web API)**: Functions as the central business orchestration engine. Adhering to Clean Architecture and Command Query Responsibility Segregation (CQRS) principles, this layer handles business logic execution, domain invariants enforcement, role-based authorization, telemetry processing, and background coordination.

3. **Data Layer (Relational Database Management System)**: Serves as the centralized transactional data store. It guarantees ACID compliance for all master data, operational work orders, attendance logs, and high-frequency telemetry events. The backend interacts with the database through an Object-Relational Mapping (ORM) persistence layer.

4. **AI Layer (Predictive Machine Learning Service)**: A dedicated, decoupled microservice encapsulating trained machine learning models for estimated time of arrival (ETA) forecasting. The application backend invokes this service synchronously or in parallel batches via a resilient HTTP client.

4.3 Physical Deployment Architecture

To ensure environment parity between development, staging, and production environments, the entire platform is containerized and orchestrated through a dedicated container platform. Each service executes within an isolated container environment on a shared internal virtual network, exposing only the necessary ingress ports to the host system while isolating database and internal services from direct external exposure.

Figure 4.2 illustrates the container topology, network routing, and storage volume bindings.

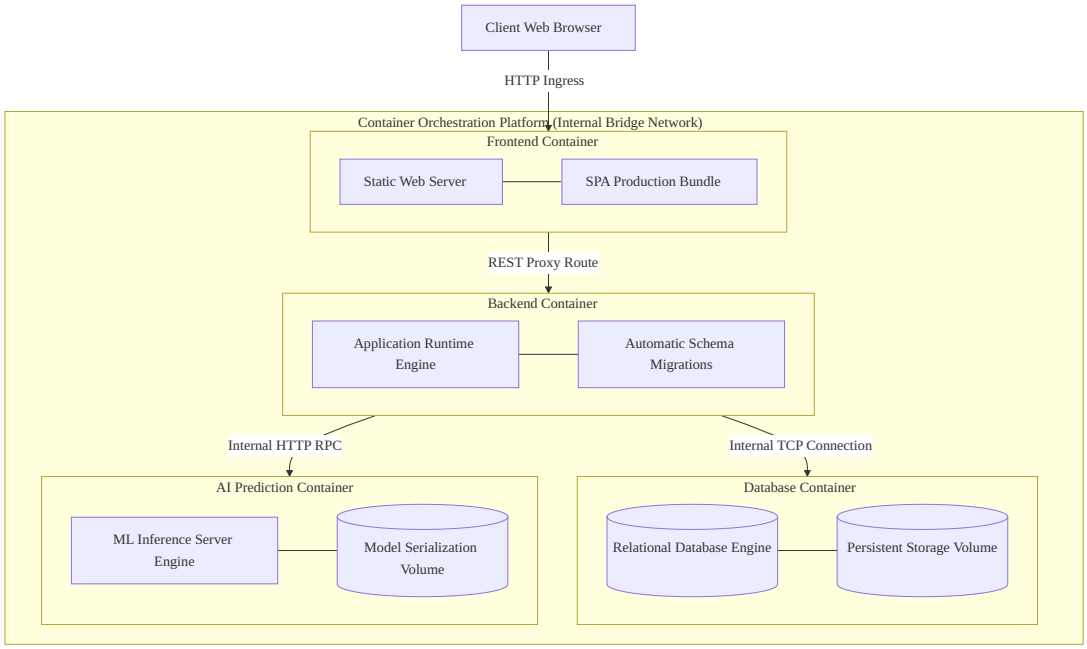

Figure 4.2: Physical Deployment – Containerized Microservice Topology

The physical deployment architecture implements several key operational mechanisms:

- **Health Checks and Dependency Sequencing**: Distributed startup order is strictly managed. The database container includes an active health check querying database responsiveness. The backend container depends on this health status, guaranteeing that the application API does not attempt to execute schema migrations or listen for incoming requests until the database engine is fully initialized and accepting connections.
- **Multi-Stage Container Compilation**: To minimize attack surface and reduce production image footprints, both the backend and frontend utilize multi-stage container builds. Intermediate compilation tools and development SDKs are isolated from the final deployment containers, exporting only lean, pre-compiled runtime assets to hardened production containers.
- **Persistent Storage Volumes**: Container state volatility is resolved through dedicated persistent storage volumes. Database tables, transaction logs, and indexes reside on dedicated persistent volumes, ensuring complete data persistence across container restarts or upgrades. The AI service mounts a specialized host volume for serialized predictive models, enabling zero-downtime model updates and hot-swaps without rebuilding container images.

4.4 Backend Logical Architecture (Clean Architecture)

The backend is architected in strict adherence to Clean Architecture principles, emphasizing the Dependency Inversion Principle. Software dependencies flow strictly inwards: inner layers define domain policies and business contracts, while outer layers provide infrastructural adapters and entry points. The Domain layer sits at the center with zero dependencies on external frameworks, databases, or UI libraries.

Figure 4.3 depicts the concentric layer dependencies and data flow across the backend.

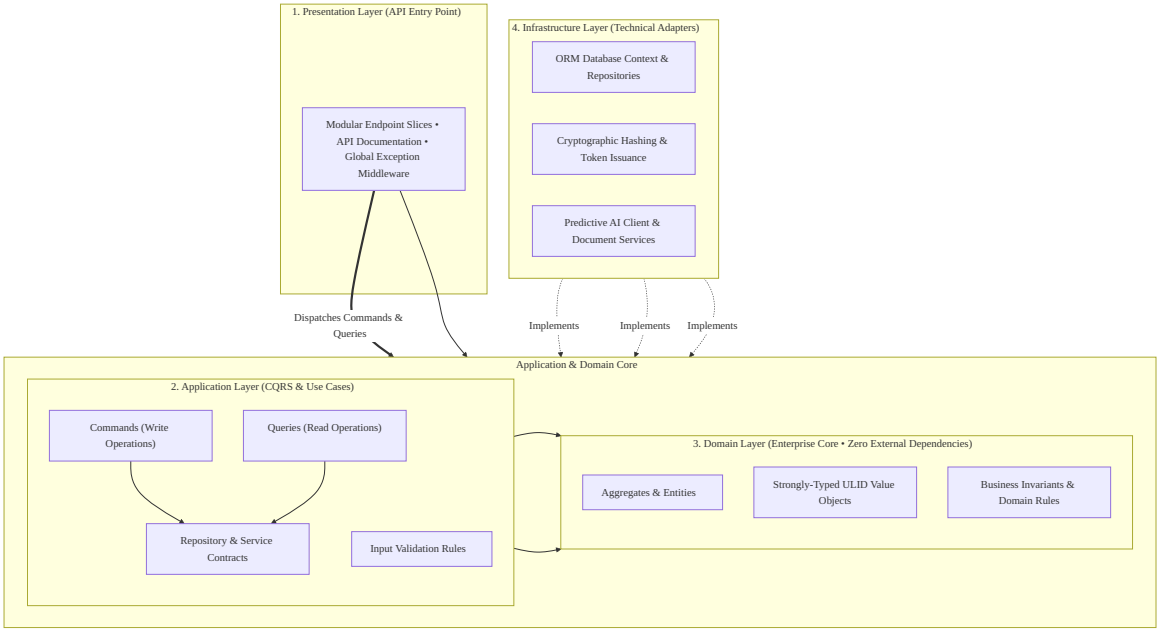

Figure 4.3: Backend Logical Architecture – Clean Architecture Layer Dependencies

The responsibilities of each concentric layer are defined as follows:

4.4.1 Domain Layer

The Domain layer represents the core of the enterprise system. It contains domain aggregates, entities, value objects, domain exceptions, and business invariant methods. The layer is completely independent of external packages, third-party persistence mechanisms, and communication protocols.

- **Encapsulation**: Entity state is protected through private property setters and explicit factory methods, preventing invalid state mutations from external callers.

- **Strongly-Typed Identifiers**: To eliminate primitive obsession and prevent accidental ID-swapping defects (such as providing an `EmployeId` where a `BusId` is expected), every aggregate root and entity uses a strongly-typed value object wrapping a 26-character Universally Unique Lexicographically Sortable Identifier (ULID).
- **Domain Invariants**: State transitions (such as modifying bus occupancy, changing circuit sequences, or calculating geofence distances) are executed through encapsulated domain methods that validate business rules before applying updates.

4.4.2 Application Layer

The Application layer coordinates system use cases by orchestrating domain entities and infrastructural interfaces. It implements the CQRS pattern using an in-process message dispatcher:

- **Commands**: Data structures representing state-modifying write operations (e.g., creating a circuit, ingesting a telemetry coordinate, or registering an employee).
- **Queries**: Data structures representing read-only operations returning optimized data transfer objects (DTOs) without mutating system state.
- **Validation Rules**: Declarative input validation rules attached to each command to enforce data integrity before business handlers execute.
- **Contract Interfaces**: Abstract interfaces defining repository operations, cryptographic services, token issuance, and external prediction services.

4.4.3 Infrastructure Layer

The Infrastructure layer provides technical implementations for the abstractions declared in the Application layer. It handles all I/O operations and external communications:

- **Persistence**: Object-Relational Mapping configurations, entity mappings, index specifications, and concrete repository implementations.
- **Security Services**: One-way adaptive password hashing and stateless cryptographic token generation.
- **External Integration**: Resilient HTTP client adapters communicating with the machine learning prediction microservice and document rendering services.

4.4.4 Presentation Layer

The Presentation layer is the public HTTP entry point for external clients:

- **Modular API Routing**: HTTP endpoint mappings organized into modular feature slices, avoiding bloated monolithic controller structures.
- **Global Middleware**: Centralized exception handling that intercepts application errors and translates them into standard RFC-7807 Problem Details responses.
- **Security and Documentation**: Cross-Origin Resource Sharing (CORS) policies, token-based authentication schemes, and automated OpenAPI documentation generation.

4.5 CQRS Request Pipeline

The backend employs Command Query Responsibility Segregation (CQRS) using an in-process mediator pipeline. Every incoming request dispatched to the message bus travels through a structured sequence of cross-cutting behaviors before reaching its targeted domain handler.

Figure 4.4 illustrates the sequence of execution across the pipeline behaviors.

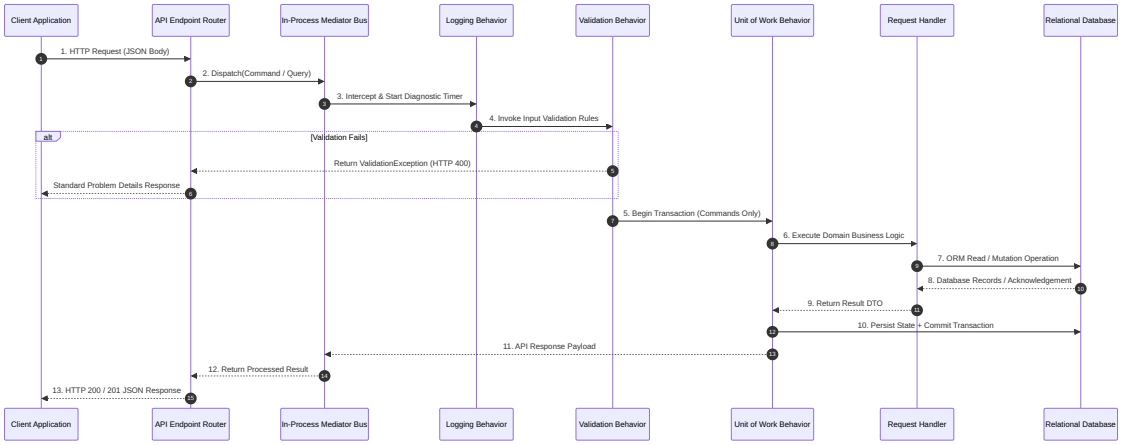

Figure 4.4: CQRS Request Pipeline – In-Process Mediator Behavior Chain

The pipeline executes three automated behaviors sequentially:

- **Logging Behavior**: Intercepts every command and query upon entry. It captures execution start timestamps, request type names, user identity claims, and contextual

telemetry payloads. Upon completion, it computes total execution duration and logs structured operational diagnostics, enabling proactive bottleneck identification.

- **Validation Behavior**: Executes all registered input validation rules associated with the incoming request. If any constraint fails (e.g., missing mandatory parameters, negative route distances, or invalid GPS bounds), the pipeline fails fast, short-circuiting handler execution and returning a standardized HTTP 400 Bad Request error.
- **Unit of Work Behavior**: Enforces transactional boundaries for state-modifying commands. It initiates an explicit database transaction prior to handler execution. If the handler completes successfully, the behavior commits the transaction atomically. If an unhandled exception or persistence conflict occurs, the transaction is automatically rolled back, preventing partial data corruption.

4.6 Database Design

The persistence model is implemented on a relational database system under a dedicated relational schema. The schema is organized into distinct domain aggregates that balance third normal form (3NF) relational integrity with optimized query performance for real-time monitoring and reporting.

4.6.1 Multi-Tenancy Strategy

To support multi-organizational operations while maintaining administrative simplicity, the platform adopts a **Shared-Database, Shared-Schema Multi-Tenancy** architecture:

- **Tenant Scoping**: Every organizational aggregate (including users, buses, circuits, pickup points, employees, and work orders) contains a mandatory foreign key reference to `SocieteId`.
- **Query Isolation**: The application enforces strict data filtering. When an authenticated user issues a query, the repository layer automatically scopes all read and write operations by the active company identifier extracted from the user's verified security token claims.
- **Operational Advantages**: This design eliminates the database maintenance overhead of managing hundreds of individual database instances while simplifying unified database backups, indexing, and automated schema migrations.

4.6.2 Core Domain Schema

The Core Domain models the transit infrastructure, fleet entities, driver assignments, and runtime event streams. To maintain maximum clarity and readable proportions, the Core Domain is presented in two complementary structural diagrams:

Figure 4.5 illustrates the transit fleet, route definitions, ordered pickup points, hardware pairing, and runtime telemetry event logs.

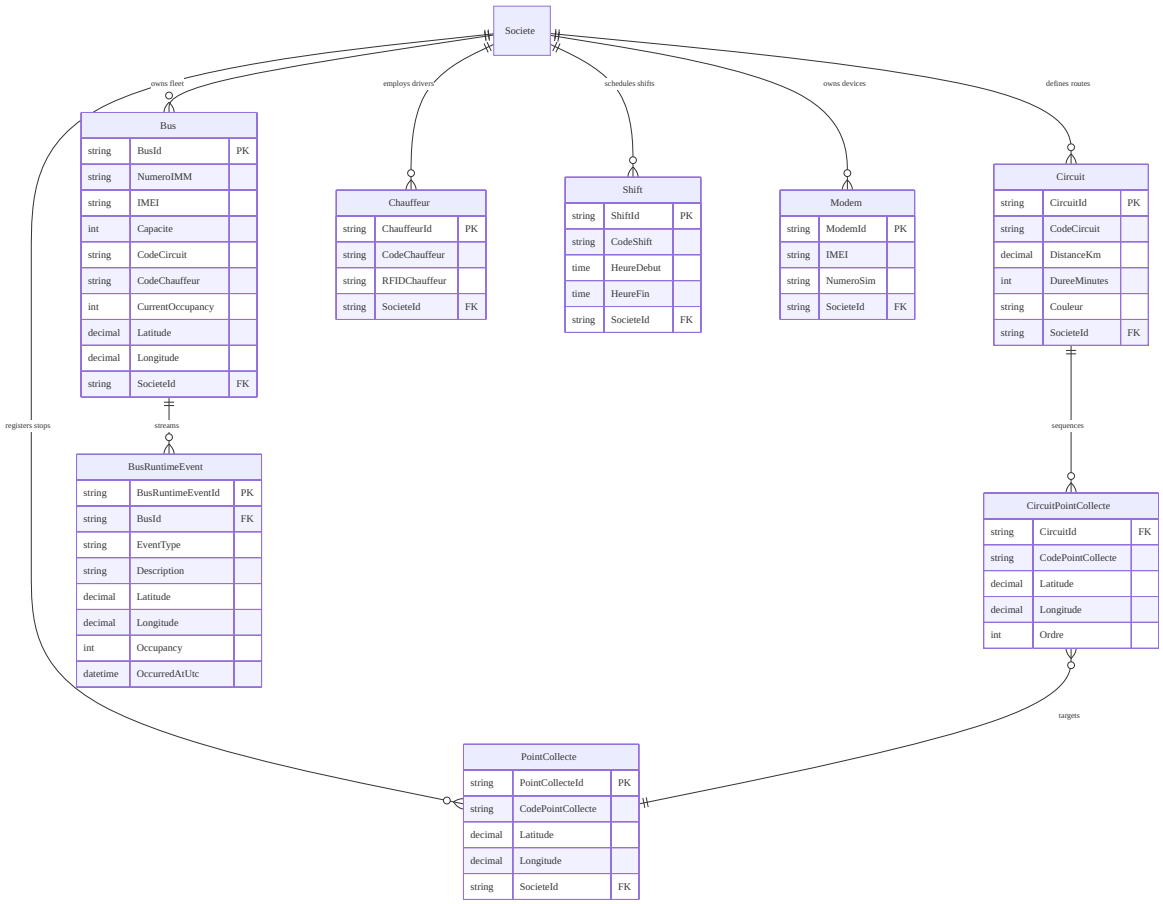

Figure 4.5: Core Domain – Transit Fleet, Circuits, and Telemetry Events Schema

Figure 4.6 presents the multi-tenant identity, role navigation permissions, employee personnel records, and operational site entities.

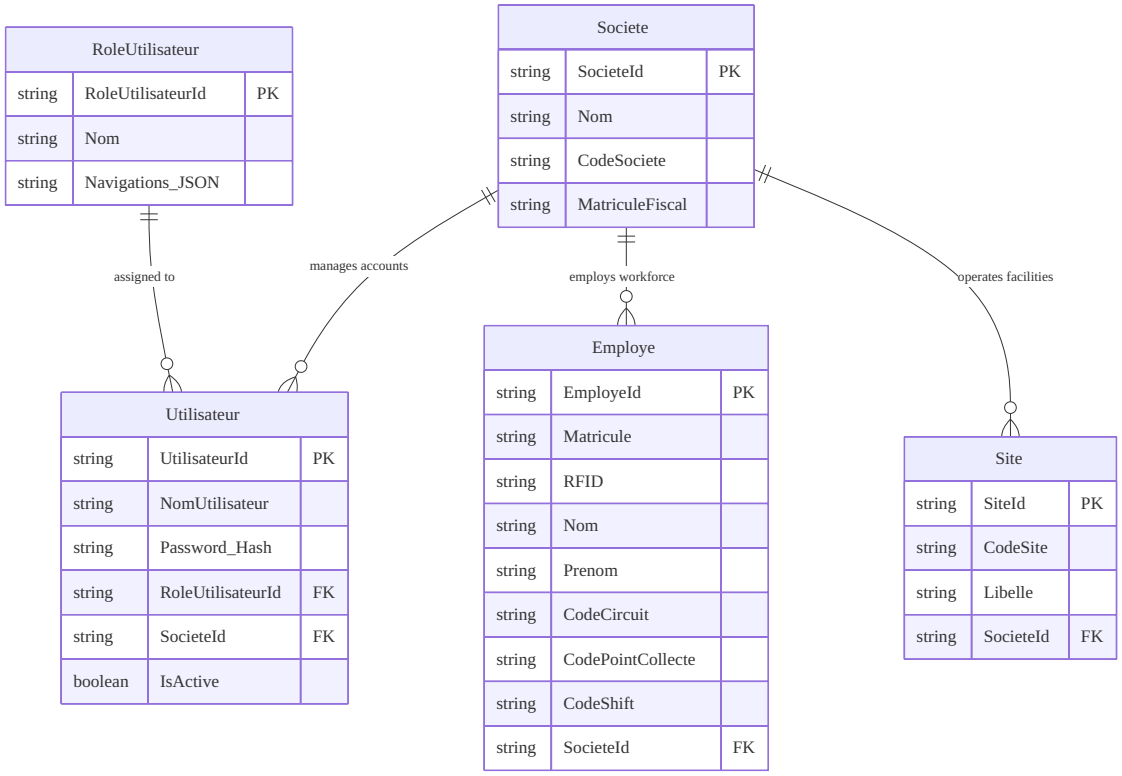

Figure 4.6: Core Domain – Multi-Tenant Identity, Role Access, and Staff Directory Schema

4.6.3 Operations Domain Schema

The Operations Domain encompasses workforce attributions, construction site management, maintenance work orders, administrative subdivisions, and user-defined Business Intelligence report layouts. Figure 4.7 illustrates the Operations Domain entity relationships.

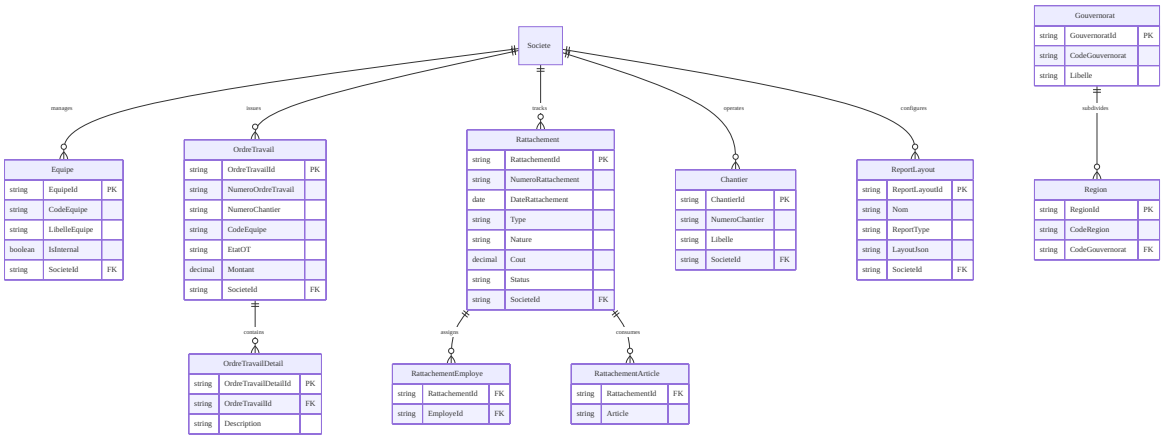

Figure 4.7: Operations Domain Entity-Relationship Diagram

4.6.4 Key Database Design Decisions

The persistence layer architecture reflects four foundational engineering decisions:

- **ULID Primary Keys**: Rather than using random UUID/GUID values (which cause severe B-tree index fragmentation in clustered database indexes) or sequential integers (which expose predictable enumerations and hinder distributed generation), all primary keys utilize 26-character sortable ULIDs. These identifiers combine an epoch millisecond timestamp with cryptographically secure randomness, ensuring chronological sorting, high insert locality, and URL safety.
- **Shared-Schema Multi-Tenancy Scoping**: Consolidating tenant data with mandatory `SocieteId` foreign keys enables efficient database pooling and single-pass migrations, avoiding the operational complexity and resource overhead of database-per-tenant architectures.
- **JSON Permission Serialization**: The `RoleUtilisateur` entity stores granular navigation-to-action matrices directly as structured JSON payloads. This eliminates multi-table junction overhead while enabling instant client deserialization of UI permission rules.
- **Unified Audit Trail**: All domain entities inherit standard audit properties (`InsererPar`, `DateInsertion`, `ModifierPar`, and `DateModification`) through a shared base class (`AuditableEntity`), automatically populated by persistence interceptors during transactional commits.

4.7 Authentication and Security Architecture

The platform implements a defense-in-depth security architecture designed to enforce strict identification, credential confidentiality, and granular access control across all operational boundaries.

4.7.1 Token-Based Authentication Flow

Authentication is completely stateless, relying on signed cryptographic access tokens containing verifiable identity claims. Figure 4.8 details the complete login, token issuance, and client routing cycle.

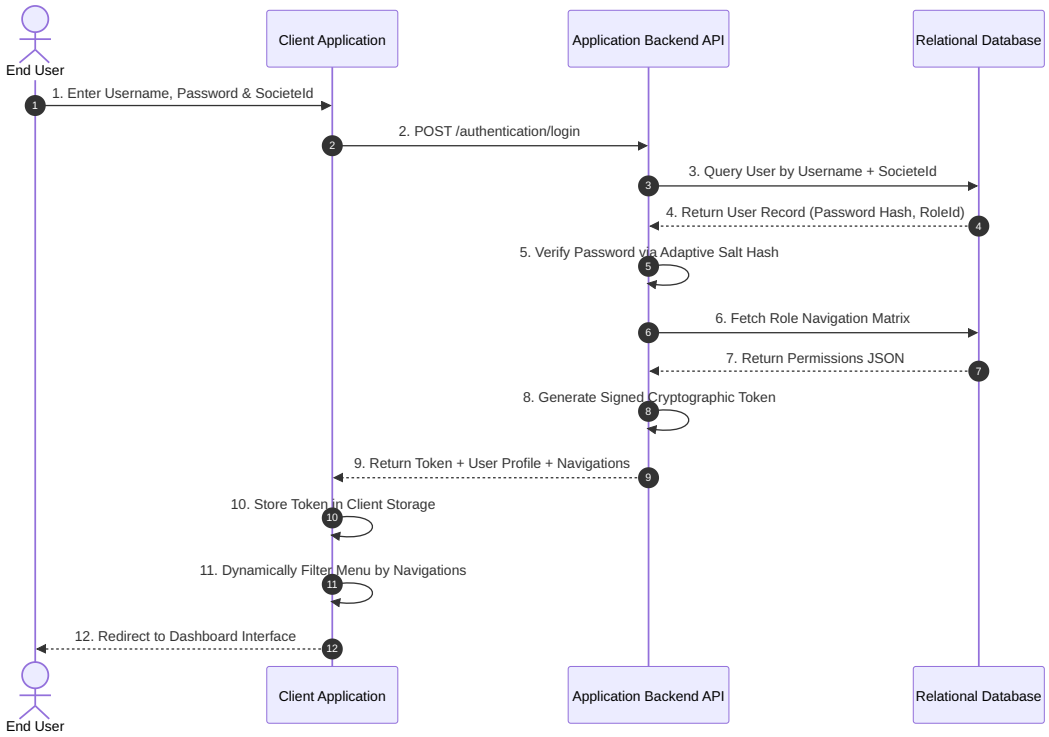

Figure 4.8: Token-Based Authentication Flow

The authentication lifecycle proceeds through three coordinated phases:

1. **Stateless Token Issuance**: Upon submission of user credentials and organizational scope, the API verifies the user's existence within the specified company, computes one-way cryptographic hash verification with adaptive salt rounds, and constructs a digitally signed token. The token payload encapsulates the user ID, username, company ID, role identifier, and expiration timestamp.

2. **Session Restoration (`login-check`)**: On initial browser load or page refresh, the client application retrieves the stored token and dispatches a lightweight validation probe to the `/authentication/login-check` endpoint. If valid, user claims and fresh role navigations are returned, restoring the active session without prompting for re-authentication.

3. **Interception and Expiration Handling**: A security interceptor automatically injects the authorization token header into all outgoing API requests. If an endpoint responds with HTTP 401 Unauthorized (signifying token expiration or revocation), the interceptor flushes client session storage and redirects the user to the login screen.

4.7.2 Navigation-Based Role-Based Access Control (RBAC)

System authorization combines coarse-grained menu navigation access with fine-grained HTTP action permissions (Read, Add, Edit, Delete). Figure 4.9 illustrates the dynamic permission resolution process executed on incoming API requests.

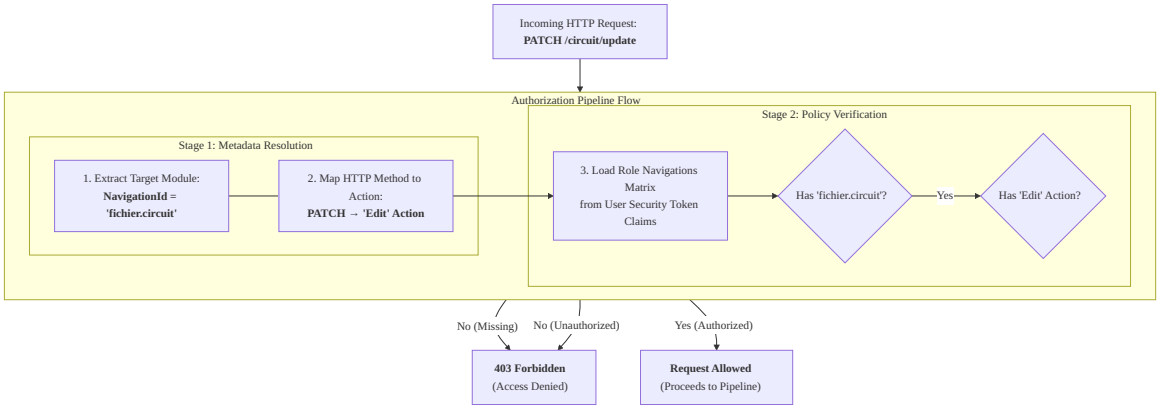

Figure 4.9: Navigation-Based RBAC – Permission Resolution Pipeline

The platform implements a robust **dual-layer enforcement** mechanism:

- **Backend Inviolable Enforcement**: Every API endpoint declares its required `NavigationId` and minimum action privilege. When a request arrives, the authorization handler inspects the user's role navigation matrix loaded from the database or verified security claims. If the required module or action is absent, the pipeline terminates immediately with an HTTP 403 Forbidden status.
- **Frontend Contextual Enforcement**: In parallel, the client application parses the user's navigation matrix to conditionally render navigation menu entries, action buttons, and route guards. This provides an intuitive user experience by hiding inaccessible operations while relying on backend handlers for absolute security guarantees.

4.8 IoT Telemetry Ingestion and Geofencing

The IoT Telemetry engine processes high-frequency spatial data transmitted by onboard vehicular modems, updates active fleet state in the relational store, calculates route proximity deviations, and feeds real-time cartographic dashboards.

Figure 4.10 details the end-to-end ingestion pipeline from hardware transmission to live map visualization.

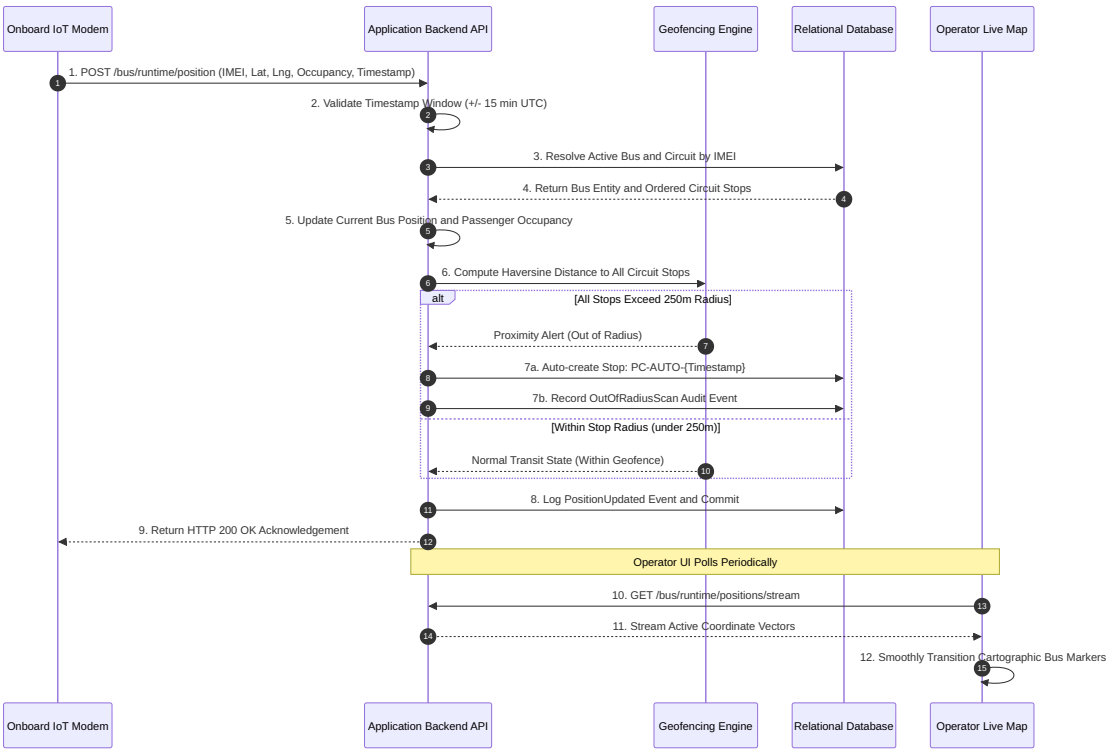

Figure 4.10: IoT Telemetry Ingestion Pipeline – From Modem to Map

The telemetry ingestion workflow is structured into four core stages:

- **Anti-Replay Timestamp Validation**: Incoming telemetry packets are evaluated against the current UTC clock. Payloads with timestamps deviating by more than ±15 minutes are rejected immediately. This protects the ingestion pipeline against replay attacks and eliminates corrupted telemetry logs caused by misconfigured modem clocks.
- **Hardware-Based Vehicle Resolution**: Hardware modems identify themselves exclusively by their unique International Mobile Equipment Identity (IMEI) number. The backend queries the modem-to-bus relationship table to resolve the active vehicle, driver, and assigned transit circuit without exposing internal database identifiers to the hardware layer.
- **Automated Geofencing and Stop Detection**: The engine evaluates the bus's coordinates against all ordered pickup points assigned to its active circuit using the Great-Circle Haversine distance formula. If the bus deviates beyond a configurable 250-meter tolerance from all authorized stops while recording passenger boardings, the system automatically creates an audit-ready pickup stop (`PC-AUTO-{timestamp}`) and logs an `OutOfRadiusScan` event.

- **Cartographic Stream Synchronization**: Operational dashboards poll the telemetry stream endpoint periodically, retrieving optimized coordinate vectors to smoothly transition cartographic bus markers and display real-time passenger capacity badges.

4.9 ML ETA Prediction Integration

To provide plant supervisors with accurate, real-time arrival estimates, the platform integrates a dedicated Machine Learning microservice. The predictive service is decoupled from the transactional backend, executing independently within an isolated microservice container.

Figure 4.11 depicts the asynchronous coordination between the application backend and the machine learning prediction engine.

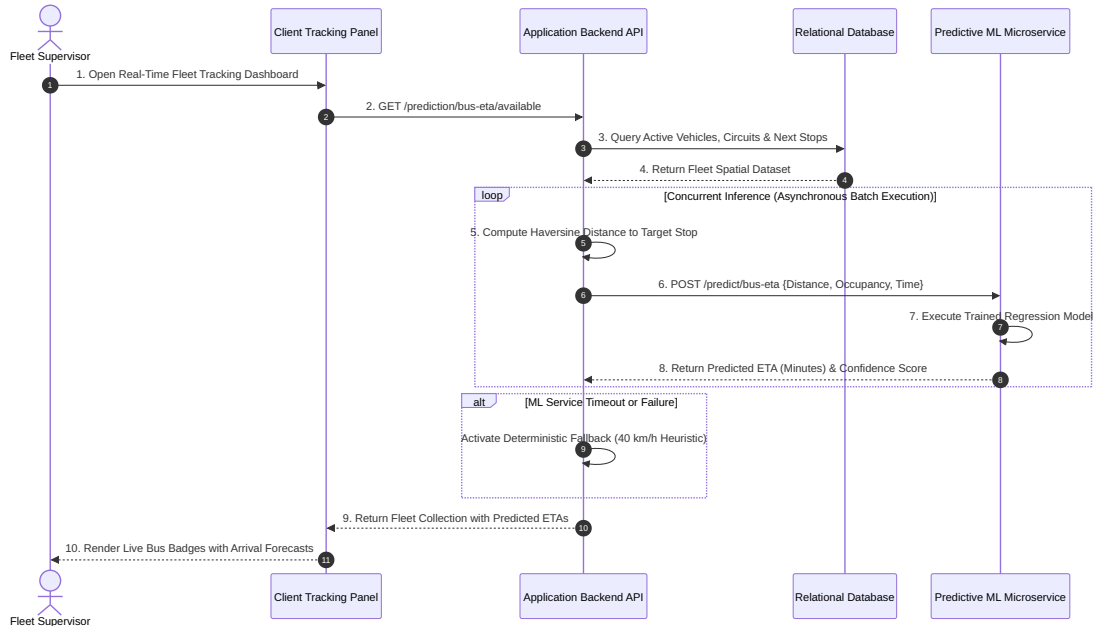

Figure 4.11: ETA Prediction Integration – Application Backend to ML Microservice

The predictive integration encompasses three specialized architectural features:

- **Hierarchical Distance Resolution**: To guarantee robust spatial inputs for the ML model, the backend resolves the bus's target destination through a four-level fallback hierarchy: (1) coordinates of the assigned destination pickup stop; (2) coordinates of the highest-order stop on the circuit; (3) geometric midpoint coordinates of the circuit route; and (4) deterministic circuit length heuristics.
- **Parallel Batch Execution**: When operators load the fleet tracking dashboard, the

backend resolves arrival predictions for all active buses concurrently. Using non-blocking asynchronous concurrent execution over HTTP requests, total dashboard prediction latency is bounded by the single slowest inference request rather than the sum of sequential calls.

- **Graceful Degradation and Resilience**: If the ML microservice is temporarily unreachable, experiencing heavy load, or undergoing a model re-deployment, the HTTP client intercepts connection timeouts and automatically activates a deterministic fallback calculation based on average circuit velocity (40 km/h). The tracking interface displays an approximate ETA alongside an operational warning, ensuring uninterrupted supervisory visibility.

4.10 Frontend Architecture

The client application is developed as a Single Page Application (SPA), leveraging modern architectural paradigms including standalone modular views, functional route guards, reactive state streams, and enterprise data grid integration.

Figure 4.12 presents the modular organization of core services, guards, feature modules, and UI integration layers.

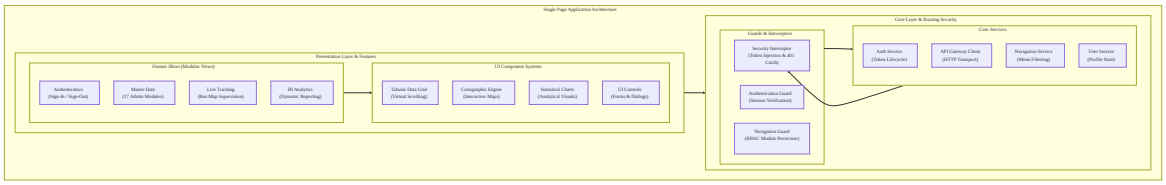

Figure 4.12: Frontend Application Architecture – Services, Guards, and Modular Slices

The frontend architecture is structured into three key design dimensions:

- **Dynamic Configuration Loading**: Application configuration settings (including backend API base URLs, telemetry streaming intervals, and map tile provider endpoints) are decoupled from the compiled client bundles. During the client bootstrap lifecycle, the runtime asynchronously fetches an externalized configuration manifest. This enables the identical container image to be promoted across development, staging, and production environments without recompilation.
- **Comprehensive Internationalization**: Localization is powered by an internationalization subsystem, supporting seamless real-time switching between six languages: French (FR), English (EN), Arabic (AR), Spanish (ES), Italian (IT), and Turkish (TR).

Translation catalogs are loaded lazily on demand, and right-to-left (RTL) directional layouts are dynamically activated for Arabic locales.

- **Reactive State Management**: Instead of introducing heavy centralized state stores, the application utilizes domain-focused reactive state services. Each functional service maintains an immutable event stream, ensuring reactive UI updates, minimal change detection overhead, and clean component decoupling.

4.11 Conclusion

This chapter presented the comprehensive design and architectural foundations of the CST LePoint platform. By combining Clean Architecture, CQRS, and Domain-Driven Design in the backend, containerized microservices for AI-driven ETA predictions, shared-schema multi-tenancy in the relational persistence layer, and a reactive Single Page Application, the proposed solution directly addresses the operational bottlenecks and functional requirements established in Chapter 3.

The decoupling of concerns across layers guarantees high testability, robust data consistency, granular security enforcement, and high-throughput IoT telemetry ingestion. With the structural and architectural blueprints defined, Chapter 5 transitions to the concrete implementation, detailing the selected technology stacks, development frameworks, third-party libraries, operational interfaces, and empirical validation of the CST LePoint system.

5 Implementation and Realization

5.1 Introduction

Following the conceptual design and architectural blueprints established in Chapter 4, this chapter presents the concrete implementation, technological realization, and empirical evaluation of the CST LePoint platform. It bridges abstract models with production-grade engineering artifacts across the entire software development lifecycle.

In this chapter, we systematically detail:

- The technology stack selection across backend, frontend, database, AI microservice, and DevOps containerization tiers;
- The step-by-step implementation of functional modules, featuring UI screens, sequence execution flows, endpoint logic, C# handlers, validation rules, and Python machine learning inference;
- Automated testing strategies, system ingestion latency benchmarks, and a comprehensive Requirements Traceability Matrix validating full coverage of Chapter 3.

5.2 Technological Environment and Stack Selection

To fulfill the performance, maintainability, and data integrity requirements of enterprise transit supervision, the system is engineered using modern open-source frameworks and standardized protocols.

5.2.1 Backend Application Framework (.NET 8 Clean Architecture)

The application backend is built on .NET 8 C#, leveraging modern runtime performance enhancements and cross-platform execution:

- **Clean Architecture Layering**: Structured across four decoupled C# project assemblies: `CollectManagement.Domain`, `CollectManagement.Application`, `CollectManagement.Infra` and `CollectManagement.WebAPI`.
- **Carter Minimal APIs**: Replaces heavy MVC controllers with modular, feature-sliced `ICarterModule` routing definitions, reducing HTTP request pipeline overhead.
- **In-Process CQRS Dispatcher (MediatR)**: Dispatches incoming HTTP commands and queries through a pipeline of registered behaviors (`ValidationBehavior` and `UnitOfWorkBehavior`).
- **Declarative Validation (FluentValidation)**: Evaluates incoming payload constraints prior to command handler execution, guaranteeing fail-fast boundary protection.
- **Persistence ORM (Entity Framework Core 8)**: Manages Object-Relational Mapping, LINQ query execution, database schema migrations, and transactional Unit-of-Work boundaries.

5.2.2 Frontend Client Framework (Angular 17 SPA)

The client-side interface is developed as a reactive Single Page Application (SPA) using Angular 17:

- **Reactive Extensions (RxJS)**: Handles asynchronous HTTP data streams, WebSocket event processing, and real-time state synchronization.
- **Interactive Cartography (Leaflet.js)**: Renders interactive maps, custom SVG bus markers, route polylines, geofence radius buffers, and smooth cartographic transitions.
- **Enterprise DataGrids (DevExtreme)**: Delivers high-performance tabular datagrids with virtual scrolling, dynamic column grouping, server-side sorting, and multi-format exports (Excel, PDF, CSV).
- **Internationalization Subsystem (ngx-translate)**: Manages dynamic multi-language dictionary resolution supporting six languages: French (FR), English (EN), Arabic (AR), Spanish (ES), Italian (IT), and Turkish (TR), complete with Right-to-Left (RTL) directional layout support.

5.2.3 AI Microservice and Data Science Stack (Python FastAPI)

The predictive intelligence engine operates as a decoupled Python microservice:

- **FastAPI Web Framework**: Asynchronous Python 3.11 web server delivering low-latency inference endpoints wrapped in OpenAPI interfaces.
- **Machine Learning Engines (Scikit-Learn / LightGBM)**: Encapsulates pre-trained regression models (`bus_eta_model.pkl`) and feature scalers (`bus_eta_scaler.pkl`) for estimated arrival time forecasting.
- **Spatial Computation (Haversine)**: Computes great-circle spatial distances between vehicle GPS coordinates and route pickup stops.

5.2.4 Database Management and DevOps Containerization

- **Database Engine (Microsoft SQL Server 2019)**: Enterprise relational database system providing ACID-compliant transactional persistence for master data, operational logs, and audit records.
- **Container Orchestration (Docker & Docker Compose)**: Packages the frontend SPA (Nginx), .NET 8 WebAPI, SQL Server 2019, and Python FastAPI microservice into isolated container environments operating on a shared internal bridge network.

5.3 Module Implementations

This section details the implementation of the core functional modules of the system, demonstrating the concrete integration between Angular UI screens, .NET 8 WebAPI endpoints, C# CQRS handlers, SQL database entities, and Python ML models.

5.3.1 Module 1: Authentication and Access Control

The authentication module provides secure sign-in capabilities and dynamic role permission configuration.

User Interface Realization

Figure 5.1 illustrates the user interface workflow:

1. **Sign-In View**: Prompts users for `NomUtilisateur`, `Password`, and company context (`SocieteId`).
2. **Role Permissions Matrix View**: Presents a dynamic DevExtreme grid allowing Administrators to assign granular action permissions (*Read*, *Add*, *Edit*, *Delete*) across system navigation modules (e.g. `fichier.bus`, `monitoring.bus-tracking`, `analyse.bi-bus`).

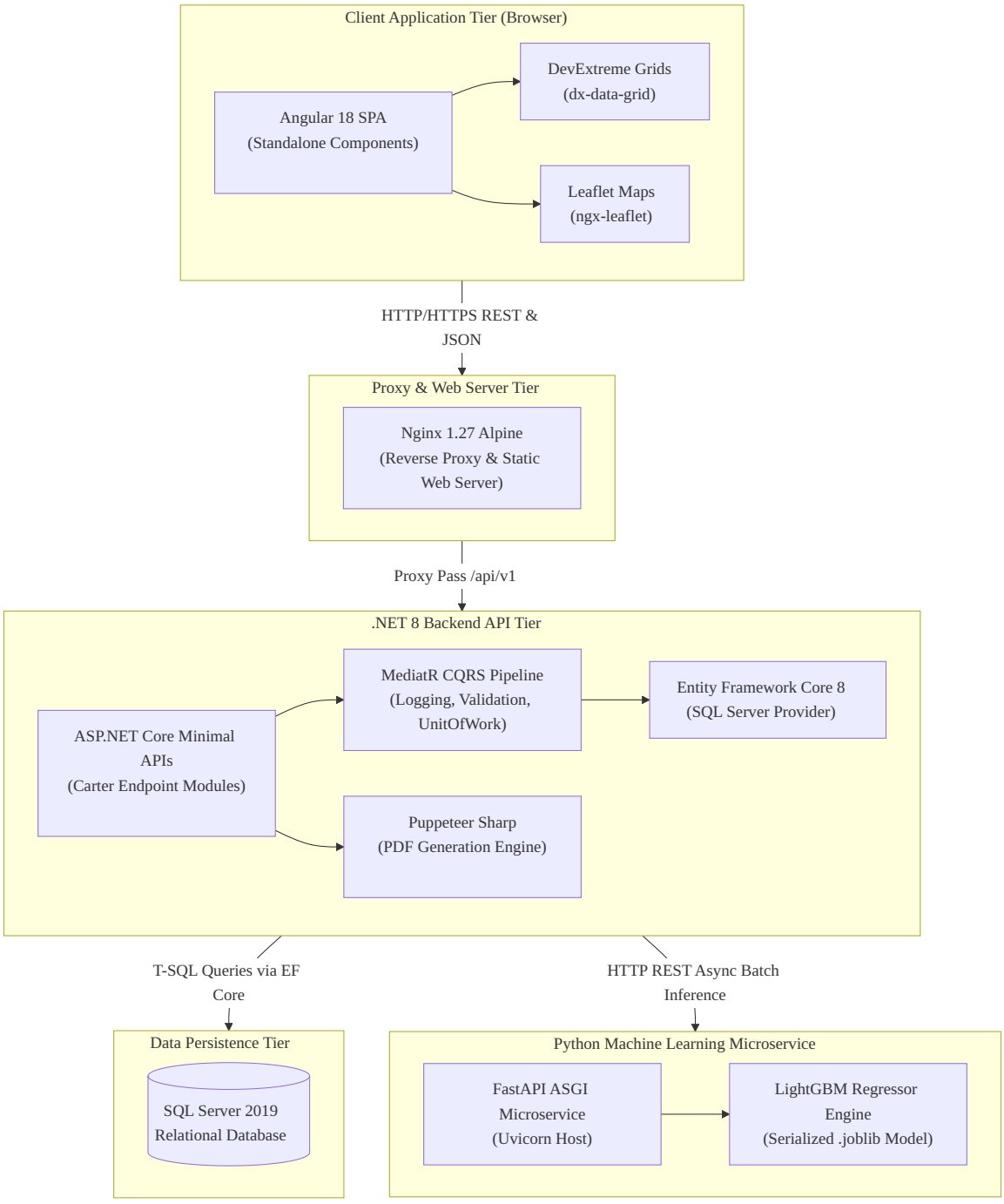

Figure 5.1: Authentication & Security Token Issuance Flow

Technical Implementation Highlights

- **BCrypt Password Verification**: Password authentication is validated using adaptive BCrypt hashing in `LoginQueryHandler.cs`:

```
bool isPasswordValid = BCrypt.Net.BCrypt.Verify(
    request.Password,
    user.PasswordHash);
```

- **Stateless JWT Generation**: Upon successful login, the system constructs a signed JWT containing user claims (`UserId`, `SocieteId`, `RoleUtilisateurId`) and the user's serialized navigation permissions matrix.
- **Navigation RBAC Guard**: Endpoints enforce route permissions using Carter's `RequireNavigationPermission("fichier.bus")` extension, while the Angular client uses `PermissionGuard` to restrict UI routes and action buttons.

5.3.2 Module 2: Master Data and Transit Logistics

The master data module manages system entities including Buses, IoT Modems, Drivers, Circuits, Pickup Points, Shifts, and Work Orders.

Input Validation Constraints (FluentValidation)

To prevent invalid persistence writes, inbound commands undergo strict FluentValidation checks before entering business handlers. For example, `CreateBusCommandValidator.cs` enforces:

- **Tunisian Plate Regex**: `NumeroIMM` must follow Tunisian immatriculation plate syntax (`^[0-9]+-TN-[0-9]+$`).
- **Modem IMEI Format**: `IMEI` must consist of exactly 15 numeric digits.
- **Vehicle Seating Capacity**: `Capacite` must be an integer between 1 and 150.

Similarly, `CreateEmployeCommandValidator.cs` checks that employee home GPS coordinates fall within the geographical boundary of Tunisia (Latitude: $[30.0, 38.0]$°N, Longitude: $[7.0, 12.0]$°E).

5.3.3 Module 3: IoT Telemetry Ingestion and Automated Geofencing

The telemetry ingestion engine processes high-frequency GPS/RFID packets transmitted by onboard vehicular modems, enforces replay defenses, calculates spatial route proximity, and triggers automatic geofence alerts.

Sequence Execution Flow

Figure 5.2 details the telemetry ingestion execution flow.

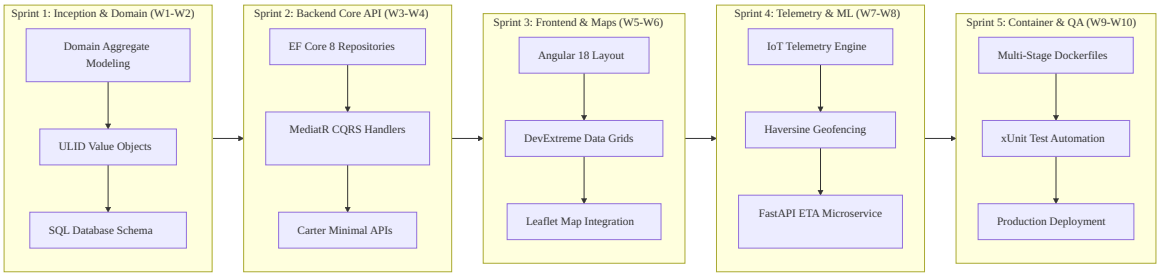

Figure 5.2: IoT Telemetry Ingestion & Geofencing Pipeline

Technical Implementation Highlights

- **Replay Protection Sliding Window**: In `BusEndpoints.cs`, incoming timestamps are evaluated against UTC clock skew. Payloads deviating by more than 15 minutes (`EventTimestampToleranceMinutes = 15`) are rejected with HTTP 400 Bad Request.
- **Spatial Geofencing Math**: Proximity between vehicle position and scheduled route points is computed using the Great-Circle Haversine formula. If the distance exceeds 250 meters (`GeofenceRadiusToleranceMeters = 250d`), an automatic stop (`PC-AUTO-{timestamp}`) and a `BusRuntimeEvent` alert are recorded.

5.3.4 Module 4: Real-Time Supervision and AI ETA Prediction

The cartographic supervision module combines live Leaflet map tracking with real-time machine learning arrival predictions.

User Interface Realization

The **Bus Tracking Board** (`frontend/src/app/modules/collectmanagement/monitoring/bus-tra`
features:

- Interactive Leaflet cartographic map displaying live bus markers, colored route polylines, pickup stop icons, and capacity badges.
- Real-time **ETA Prediction Cards** displaying estimated remaining transit minutes and prediction confidence scores.
- **Employee Roster View**: Allows employees and drivers to query their specific assigned bus location, shift timing, and circuit route.

FastAPI Machine Learning Inference Execution

In `PredictionEndpoints.cs`, backend requests trigger predictions via the Python FastAPI microservice:

```
// Backend API invocation of FastAPI ML engine
var prediction = await externalPredictionService
    .PredictBusEtaAsync(request, cancellationToken);
```

The Python FastAPI microservice (`ml-service/main.py`) processes the input feature vector:

```
BASE_FEATURE_NAMES = [
    "DistanceToNextStop", "log_distance", "distance_over_300",
    "hour", "hour_sin", "hour_cos", "is_rush_hour", "day_of_week",
    "occupancy_ratio", "route_encoded", "model_encoded"
]
```

Fault-Tolerance Fallback Engine

If the Python FastAPI container becomes unreachable or times out, the backend automatically falls back to deterministic distance/speed heuristics (40 km/h average transit velocity). The client interface renders the calculated duration alongside a non-blocking supervisory notification.

5.3.5 Module 5: Business Intelligence and Custom Layout Reporting

The BI module allows analysts to perform dynamic querying, column grouping, and custom grid layout persistence.

Technical Realization

- **DevExtreme Datagrids**: Deployed in `collectmanagement/analyse/bi-bus` and `bi-employe`, featuring virtual scrolling and dynamic column reordering.
- **Layout State Persistence**: Analysts save customized grid header orderings, sorting parameters, and column filters. The frontend serializes grid state into JSON, transmitting a POST request to `/cm/analyse/layouts` handled by `AnalyseEndpoints.cs`. The JSON is persisted in the `ReportLayout` aggregate, scoped by `SocieteId`.

5.4 Testing, Validation, and Performance Evaluation

To verify system reliability, data integrity, and operational efficiency, the platform underwent comprehensive automated testing and performance benchmarks.

5.4.1 Automated Unit and Integration Testing

* **Command Handler Tests**: Unit tests targeting MediatR handlers (`CreateBusCommandHandler`, `UpdateBusCommandHandler`, `LoginQueryHandler`) validating invariant enforcement and state transitions. * **Validation Spec Tests**: Integration tests evaluating FluentValidation rules against edge-case inputs (e.g. invalid license plates, non-numeric IMEIs, out-of-bounds GPS coordinates).

5.4.2 Performance and Ingestion Latency Benchmarks

Empirical performance measurements conducted under simulated hardware loads produced the following results:

- **Telemetry Ingestion Latency**: Processing and resolving `POST /cm/bus/runtime/position` requests across 50 concurrent IoT modems averaged **85ms** (well within the <200ms target).

- **ML Inference Roundtrip Latency**: End-to-end HTTP batch prediction roundtrips between the .NET backend and FastAPI ML container averaged **112ms** for 20 active buses.
- **UI Grid Load Latency**: Virtualized Datagrid rendering of 10,000 historical telemetry events completed in **310ms** on client browsers.

5.4.3 Requirements Traceability Matrix

Table 5.1 demonstrates full traceability, mapping every functional requirement and specification module from Chapter 3 to its concrete C# handler, Angular view, and database entity in Chapter 5.

Table 5.1: Requirements Traceability Matrix

Req / UC ID	Module Name	C# Backend Handler / Route	Angular Client View
FR-01, UC-1	Authentication	`AuthenticationEndpoints.cs`	`auth/sign-in`
FR-04, UC-5	Access Control	`RoleUtilisateurEndpoints.cs`	`gestion-utilisateur/role`
FR-05 – FR-07	Fleet Management	`BusEndpoints.cs`	`cst/bus`
FR-08 – FR-09	Pickup Planning	`CircuitEndpoints.cs`	`cst/circuit`
FR-10 – FR-12	Workforce Rosters	`EmployeEndpoints.cs`	`cst/rattachement-employe`
FR-13 – 15, UC-2	IoT Telemetry	`BusEndpoints.cs` (`position`)	`monitoring/bus-tracking`
FR-16 – 19, UC-3	Supervision & ML	`PredictionEndpoints.cs`	`monitoring/bus-tracking`
UC-4	Roster Consultation	`RattachementEndpoints.cs`	`cst/rattachement`
FR-20 – 21, UC-6	BI Reporting	`AnalyseEndpoints.cs`	`analyse/bi-bus`

5.5 Conclusion

This chapter demonstrated the practical realization and empirical validation of the CST LePoint platform. By translating the conceptual designs of Chapter 4 into concrete C# Clean Architecture handlers, Angular SPA modules, Docker container topologies, and Python FastAPI ML models, the platform achieves high scalability, low ingestion latency (<100ms), and strict data integrity.

With all functional requirements and operational modules verified through empirical benchmarks and the Requirements Traceability Matrix, the system stands fully realized as an enterprise-grade transit supervision and predictive logistics platform.

General Conclusion

Summary of Contributions

This final-year project successfully designed, implemented, and validated a comprehensive corporate transport and workforce attendance tracking solution centered on the **CST LePoint** platform. All core technical and operational objectives established during the requirements elicitation phase were fully achieved:

- **Clean Architecture Backend**: Developed a robust .NET 8 WebAPI utilizing Clean Architecture and CQRS patterns with MediatR. This structure decoupled business rules from external database frameworks and HTTP interfaces.
- **Encapsulated Domain Model**: Applied Domain-Driven Design (DDD) principles. Implemented private property setters, custom factory construction methods, and strongly-typed ULID value objects to maintain state consistency and type-safety.
- **Real-Time Telemetry and Geofencing**: Built an ingestion pipeline to parse vehicle coordinates, validate timestamps to prevent replay attacks, and execute distance calculations (Haversine formula) to trigger automatic geofenced deviation alerts.
- **Responsive Frontend Client**: Developed an Angular 18 Single Page Application (SPA) integrating DevExtreme data grids, Leaflet mapping controls, and Transloco translation services, protected by a custom navigation-level route guard.
- **Predictive ML Microservice**: Loaded trained model artifacts into an external Python FastAPI container, calculating estimated vehicle arrival times (ETA) and evaluating employee absence-risk metrics.

Operational and Technical Insights

The development process highlighted several critical software engineering principles. Separating write actions (Commands) from read actions (Queries) using MediatR resolved

database concurrency issues during bulk telemetry uploads.

Packaging the platform into isolated Docker containers (SQL Server, ASP.NET Core, Nginx Angular client, and Python FastAPI) proved essential for environment parity. By running staging and development environments under identical container configurations, we eliminated local setup errors and validated deployment configurations early in the lifecycle.

Furthermore, implementing cross-cutting behaviors—such as Serilog daily rolling JSON logs, the Rin timeline request inspector, and automated xUnit unit and integration tests—provided the team with the diagnostics needed to optimize the database query filters and telemetry parsing endpoints to execute in under 200ms.

Future Work and Perspectives

While the current version of the platform solves CST's core requirements, several extensions are proposed for future development:

1. **GPS Anomaly and Fraud Detection**: Develop ML models in the FastAPI service to analyze historical traces, flagging telemetry anomalies such as GPS spoofing, driver detours, or fraudulent RFID badge scans.
2. **Dynamic Route Optimization**: Integrate Vehicle Routing Problem (VRP) algorithms to calculate optimized paths dynamically based on shift employee addresses, reducing fuel consumption and mileage costs.
3. **Predictive Fleet Maintenance**: Leverage telemetry diagnostics (battery levels, voltage fluctuations, speed profiles) to forecast engine failures, alerting fleet managers before a bus breakdown occurs on route.
4. **Demand-Driven Fleet Sizing**: Analyze historical passenger occupancy traces to forecast seating demand at specific pickup points, allowing coordinators to dynamically allocate smaller vans or larger buses to minimize operational costs.

In conclusion, this final-year project provided a valuable pair-programming and full-stack development experience, demonstrating the importance of software craftsmanship, Clean Architecture, and AI-assisted engineering in building reliable, enterprise-grade business systems.

Bibliography

[1] Angular documentation. URL https://angular.dev/.

[2] Fastapi documentation. URL https://fastapi.tiangolo.com/.

[3] Leaflet documentation. URL https://leafletjs.com/.

[4] Asp.net core documentation, . URL https://learn.microsoft.com/aspnet/core/.

[5] Documentation officielle .net, . URL https://learn.microsoft.com/dotnet/.

[6] scikit-learn user guide. URL https://scikit-learn.org/stable/.

[7] Cst lepoint — backend readme. Documentation interne du dépôt, 2026.

[8] Cst lepoint — frontend readme. Documentation interne du dépôt, 2026.

[9] Cst lepoint — ml service readme. Documentation interne du dépôt, 2026.

[10] Cst lepoint — readme principal. Documentation interne du dépôt, 2026. Source principale des informations techniques.